Neurocomputing 391 (2020) 96–118

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

Physics Informed Extreme Learning Machine (PIELM)–A rapid method for the numerical solution of partial differential equations

Vikas Dwivedi*, Balaji Srinivasan

Heat Transfer and Thermal Power Laboratory, Department of Mechanical Engineering, Indian Institute of Technology Madras, Chennai 600036, India

ARTICLE INFO

Article history:
Received 27 June 2019
Revised 24 September 2019
Accepted 23 December 2019
Available online 27 December 2019

Communicated by Prof. Zidong Wang

Keywords:
Partial differential equations
Physics informed neural networks
Extreme learning machine
Advection-Diffusion equation

ABSTRACT

There has been rapid progress recently on the application of deep networks to the solution of partial differential equations, collectively labeled as Physics Informed Neural Networks (PINNs). In this paper, We develop Physics Informed Extreme Learning Machine (PIELM), a rapid version of PINNs which can be applied to stationary and time-dependent linear partial differential equations. We demonstrate that PIELM matches or exceeds the accuracy of PINNs on a range of problems. We also discuss the limitations of neural network-based approaches, including our PIELM, in the solution of PDEs on large domains and suggest an extension, a distributed version of our algorithm – DPIELM. We show that DPIELM produces excellent results comparable to conventional numerical techniques in the solution of time-dependent problems. Collectively, this work contributes towards making the use of neural networks in the solution of partial differential equations in complex domains as a competitive alternative to conventional discretization techniques.

1. Introduction

Partial differential equations (PDEs) are widely used in the mathematical modeling of various physics, engineering and finance problems. In practical situations, these equations typically lack analytical solutions and are solved numerically. In current practice, most numerical approaches to solve PDEs like Finite Element Method (FEM) [1], Finite Difference Method (FDM) [2] and Finite Volume Method (FVM) [3] are mesh-based. A typical implementation of a mesh-based approach involves three steps: (1) the generation of grids, (2) the discretization of the governing equation, and (3) the solution of discrete equations with a particular iterative process. Some of the problems inherently associated with these methods are as follows:

1. In complex computational domains, they can not be used to solve PDEs because grid generation itself becomes infeasible.
2. The discretization process creates a discrepancy between the mathematical nature of actual PDE and its approximate difference equation [3]. Sometimes this can lead to quite serious problems [4].

One of the options to fix these issues is to use neural networks. In this approach, the sampling points consist of some randomly selected locations inside the domain and on the boundary. The neural network is used as the basis function to approximate the solution of PDE, and thus PDE. The problem of solving the PDE is treated as an optimization problem in which losses due to neural network approximations of PDE and boundary conditions (BCs) are minimized.

There are three primary motivations for this approach. First, being universal approximators [5], neural networks can potentially represent any continuous function. Second, as the partial derivatives of the neural network solution can be found in closed form using automatic differentiation [6], neural networks can also represent any PDE. Finally, this method is meshfree, and therefore complex geometries can be easily handled [7].

Initial work in this direction can be credited to Lagaris et al. [8,9] Firstly, they solved the initial boundary value problem using neural networks, and later they extended their work to handle irregular boundaries. Since then, many researchers have contributed to solving initial and boundary value problems in arbitrary geometries [10–13].

Among various deep neural network-based approaches, the physics-informed learning machines have recently produced promising results for a series of nonlinear benchmark problems. The central idea of this approach is to include prior-structured information about the solution in the learning procedure. The initial promising results using this method are reported in the references [14–16]. In these papers, the authors have employed Gaussian process regression to accurately infer the solutions to linear and

* Corresponding author. Tel.: 7358441926.
E-mail addresses: vikas.dwivedi90@gmail.com, me15d080@smail.iitm.ac.in (V. Dwivedi), sbalaji@iitm.ac.in (B. Srinivasan).

https://doi.org/10.1016/j.neucom.2019.12.099
0925-2312/

nonlinear problems along with the associated uncertainty estimates. This approach was later extended to both inference and system identification problems [17,18]. Finally, the physics informed neural networks (PINN) were introduced [19] for both data-driven solutions of PDEs as well as the data-driven discovery of parameters in a PDE.

Recently, Berg et al. [7] have developed a PINN based method to solve PDEs on complex domains and produced several good results. Despite various advantages of using deep networks for solving PDEs, PINNs have several problems [19]. Firstly, the learning process of PINNs is hand-tuned. For example, there is no way to determine how much data or which architecture is good enough for a given choice of sampling points. Then, there is no guarantee that the algorithm will not hit upon a local minimum or will not suffer from the problem of vanishing gradients for very deep architectures. Finally, they are much slower than traditional numerical methods. Therefore, their slow learning speed makes them unfit for practical problems.

We show that some of the problems mentioned above can be easily handled by using an alternative network called the Extreme Learning Machine (ELM). The basic ELM was proposed by Huang et al. [20] for a Single Layer Feedforward Networks (SLFNs), and later it was extended to generalized SLFNs. The essence of ELM is that the weights of the hidden layer of SLFNs need not be learned. It has been shown that ELM is much faster than the typical deep neural networks that are based on gradient-descent type optimization methods. Therefore it alleviates the learning speed problem. Previously, ELMs have been used in approximating functions [21] and solving ordinary differential equations (ODEs) and stationary PDEs [22,23] using Legendre and Bernstein polynomial basis functions respectively.

In this paper, we propose a new machine-learning algorithm to solve stationary and time-dependent PDEs in complex geometries. We have named it physics informed extreme learning machine (PIELM) because it is a combination of two algorithms, namely ELM and PINN. Theoretically, there is no question over the employment of ELM as a PDE solver because it is a universal approximator [24], and therefore it can approximate any PDE. We have made our ELM "physics informed" by incorporating the information about the physics of PDE as the cost function. In addition to this, we have also proposed an extension to original PIELM called distributed PIELM that enhances the representation power of PIELM without adding any extra hidden layers. We demonstrate that both PIELM and DPIELM exhibit superior performance on a range of stationary and time-dependent problems in comparison to existing gradient-based methods.

This paper proceeds as follows. Section 2 gives a brief review of PINN and ELM. We propose the PIELM in Section 3. Performance evaluation on various stationary and time-dependent PDEs is given in Section 4. Section 5 discusses the limitations of both PIELM and PINN in solving PDEs that admit sharp local gradient solutions. To overcome this limitation, we propose DPIELM, the distributed version of PIELM in Section 6. In Section 7, we evaluate the performance of DPIELM on the pathetic cases where the original PIELM failed. In particular, we demonstrate for the first time that an ELM based learning algorithm can be used to solve the 2D unsteady advection-diffusion equation in low viscosity regime accurately. The paper ends with conclusions and brief details of future work in Section 8.

2. Brief review of ELM and PINN

The PIELM is a combination of two learning algorithms: ELM and PINN. In this Section, we review these two algorithms in brief.

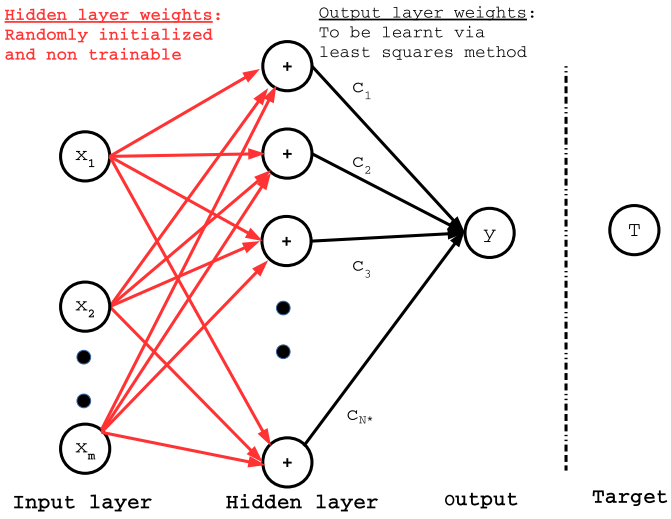


Fig. 1. Basic structure of ELM

2.1. Extreme learning machine

Traditional gradient-based learning algorithms [25] have prohibitively slow learning speed, and they suffer from various problems like improper learning rate, local minima, etc. Huang et al. [24] originally proposed a novel learning algorithm called ELM to fix these issues. ELM is a single layer feed-forward neural network which randomly assigns input layer weights and analytically determines the output weights. ELM is thousands of times faster [24] and mostly shows better generalization performance than gradient-based learning approaches like backpropagation. A typical implementation of the algorithm involves the following steps:

1. Select a shallow neural network.
2. Fix the hidden layer weights and biases randomly. These parameters will not be learned, and therefore no iterative optimization is required for them.
3. Apply a nonlinear transformation to the input data set. It gives the input to the final layer.
4. Take the linear combination of all the inputs of the final layer. It is the output of ELM.
5. Learn the output layer weights using the least-squares method.

2.2. Mathematical formulation

Consider the basic ELM shown in fig. 1. It is a single layer feed forward neural network with N^* neurons in the hidden layer. $\vec{x}$ is the input vector of size $m \times 1$ and y is the output. We denote the non-linear activation by φ and the weights and biases of j^{th} node of hidden layer by $\vec{w}_j$ and $b_j$respectively. The size of $\vec{w}_j$ is the same as $\vec{x}$. The output of the hidden node j is $\varphi(\vec{w}_j^T \vec{x} + b_j)$. For a given data set $\{(\vec{x}_k, y_k)\}_{k=1}^N \subset \Re^m \mathrm{x} \Re^1$ with N distinct samples, the ELM output is given by

$$\vec{y} = \boldsymbol{H}(\vec{x})\vec{c} \tag{1}$$

where,

$$\boldsymbol{H} = [\vec{h}(\vec{x}_1), \vec{h}(\vec{x}_2), ..., \vec{h}(\vec{x}_N)]^T,$$

$$\vec{h}(\vec{x}_k) = [\varphi(\vec{w}_1^T \vec{x}_k + b_1), \varphi(\vec{w}_2^T \vec{x}_k + b_2), ..., \varphi(\vec{w}_{N^*}^T \vec{x}_k + b_{N^*})],$$

and $\vec{c} = [c_1, c_2, ..., c_{N^*}]^T$ is the vector of output layer weights.

Note that if the number of hidden layer nodes is equal to the number of training samples, i.e. $N^* = N$, ELM can approximate

these training samples with zero error. Therefore, for randomly assigned and non-trainable input layer weights, the ELM prediction ($\vec{c}$) comes from least squares solution a system of linear equation given below.

$$\boldsymbol{H}\vec{c} = \vec{T} \tag{2}$$

However, for a non-square $\boldsymbol{H}$, the approximation error ($\vec{\varepsilon} = \vec{y} - \vec{T}$) will not be zero. Therefore, for such problems, we define the cost function J as the mean square of the residual, i.e.

$$J = \frac{1}{2N}\vec{\varepsilon}^{T}\vec{\varepsilon}. \tag{3}$$

In order to minimize J, we set all the directional derivatives to zero $\left(\frac{\partial J}{\partial c_k} = 0\right)$, which gives us the normal equations. These equations are solved analytically using pseudo inverse. Therefore, the ELM solution for the outer layer weights is given by

$$\vec{c} = pinv(\boldsymbol{H})\vec{T} \tag{4}$$

where $pinv(\boldsymbol{H})$ is the pseudo inverse of $\boldsymbol{H}$.

2.3. Physics informed neural network

Raissi et al.[19] proposed PINN for approximating solutions to general nonlinear PDEs as well as some related inverse problems and validated it with a series of benchmark test cases. Unlike conventional mesh-based methods that use piece-wise polynomials, PINN uses a single deep neural network (DNN) as the basis function. The PDE and the boundary conditions (BC) are sampled on randomly located sampling points within and at the boundary of the computational domain, respectively. The output of the DNN is used to calculate the residuals of the known PDE and associated boundary conditions. The weights and the biases of the DNN are learned by minimizing the physics-based cost function, i.e., the summation of PDE and BCs least-squares cost using any gradient-based optimization method. The embedding of the information about the PDE and BC in the cost function is the main feature of PINN. We explain this using a simple example of a 1D unsteady equation in the next sub-section.

2.4. Mathematical formulation

Consider the following 1D unsteady problem

$$\frac{\partial}{\partial t}u(x,t) + \mathcal{N}u(x,t) = R(x,t),\ (x,t) \in \Omega \tag{5}$$

$$u(x,t) = B(x,t),\ (x,t) \in \partial\Omega, \tag{6}$$

$$u(x,0) = F(x), x \in (x_L, x_R), \tag{7}$$

where $\mathcal{N}$ is a potentially nonlinear differential operator and $\partial\Omega\ (= \partial\Omega_L \bigcup \partial\Omega_R)$ is the boundary of computational domain Ω as shown in fig. 2. It also shows the PINN with collocation points $(\vec{x}_f, \vec{t}_f)$ at the interior (red triangles) and the boundary points $(\vec{x}_{bc}, \vec{t}_{bc})$ at the four faces (blue rectangles).

We approximate $u(x, t)$ with the output $\boldsymbol{f(x, t)}$ of the PINN. The network architecture may be shallow or deep depending upon the non-linearity $\mathcal{N}$. If we denote the errors in approximating the PDE, BCs and IC by $\vec{\xi}_f$, $\vec{\xi}_{bc}$ and $\vec{\xi}_{ic}$ respectively. Then, the expressions for these errors are as follows:

$$\vec{\xi}_f = \frac{\partial \vec{f}}{\partial t} + \mathcal{N}\vec{f} - \vec{R},\ on\ (\vec{x}_f, \vec{y}_f). \tag{8}$$

$$\vec{\xi}_{bc} = \vec{f} - \vec{B},\ (\vec{x}_{bc}, \vec{t}_{bc})_{side\ faces} \tag{9}$$

$$\vec{\xi}_{ic} = \vec{f}(.,0) - \vec{F},\ (\vec{x}_{bc}, \vec{t}_{bc})_{bottom\ face} \tag{10}$$

For shallow networks, $\frac{\partial \vec{f}}{\partial t}$ and $\mathcal{N}\vec{f}$ can be determined using hand calculations. However, for deep networks, we have to use finite difference methods or automatic differentiation [6]. The latter is preferred for its computational efficiency. We can recast the PDE, BC, IC system to an optimization problem by minimizing an appropriate loss function. The loss function J to be minimized for a PINN is given by

$$J = \frac{\vec{\xi}_f^{\,T}\vec{\xi}_f}{2N_f} + \frac{\vec{\xi}_{bc}^{\,T}\vec{\xi}_{bc}}{2N_{bc}} + \frac{\vec{\xi}_{ic}^{\,T}\vec{\xi}_{ic}}{2N_{ic}}, \tag{11}$$

where N_f, N_{bc} and N_{ic} refer to number of collocation points, boundary condition points in left and right faces and initial condition points at the bottom face respectively. We can see that we have chosen a least square loss function. Now any gradient based optimization routine may be used to minimize J. The weights of the network that result are in essence the parameters that decide the representation of the solution of the PDE. *Learning* therefore is the process of converging to these parameters. A schematic of learning procedure for PINN is given fig. 3. In the figure, W_i, b_i are the weights or parameters of the PINN, i.e., they represent the parametrization of the nonlinear *basis* of the PDE.

Please note that we have shown the formulation for the 1D unsteady problem on a rectangular geometry for the sake of clarity. In principle, there are no restrictions on the shape of the computational domain or the higher dimensions of PDE. The key steps in its implementation are as follows:

1. Identify the PDE to be solved along with the initial and boundary conditions.
2. Decide the architecture of PINN.
3. Approximate the correct solution with PINN.
4. Find expressions for the PDE, BCs, and IC in terms of PINN and its derivatives.
5. Define a loss function that penalizes for error in PDE, BCs and IC.
6. Minimize the loss with gradient-based algorithms.

3. Proposed PIELM

PIELM is a hybrid neural network-based method that integrates two ideas coming from PINN and ELM (Extreme Learning Machine) separately. The first idea, coming from PINN, is to encapsulate the physics of PDE in the loss function of the machine learning algorithm. The other one, coming from ELM, is to update the weights of the outer layer only, leaving input layer weights unchanged. Since one of the chief difficulties that we found in the implementation of PINN was its slow learning, we integrate PINN with ELM to combine their advantages. References [20] and [24] describe the theoretical details and universal approximation properties of ELM respectively. Since ELMs also obey universal approximation theorem, we do not lose out on any representation power by using PIELM instead of PINN. Previously, ELMs have been used in approximating functions [21] and solving ordinary differential equations [22] and stationary PDEs [23] using Legendre and Bernstein polynomial basis functions respectively.

For a single-layer feed-forward network, if we use random initialization for the input layer weights, define a physics-based loss function, and calculate the outer layer weights analytically, the resulting neural network is called PIELM. Similar to its parent algorithms, PILEM is both data-efficient and fast as compared to the iterative gradient descent-based methods. The mathematical formulation for a simple 1D unsteady PDE is given below.

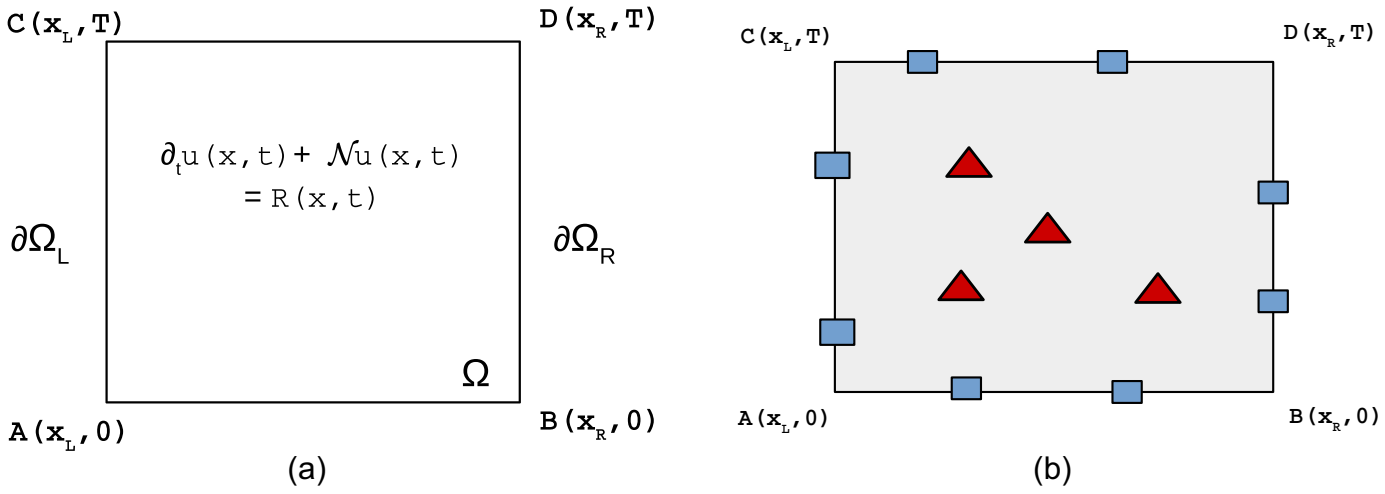


Fig. 2. A diagram of the problem: PDE in a rectangular domain.

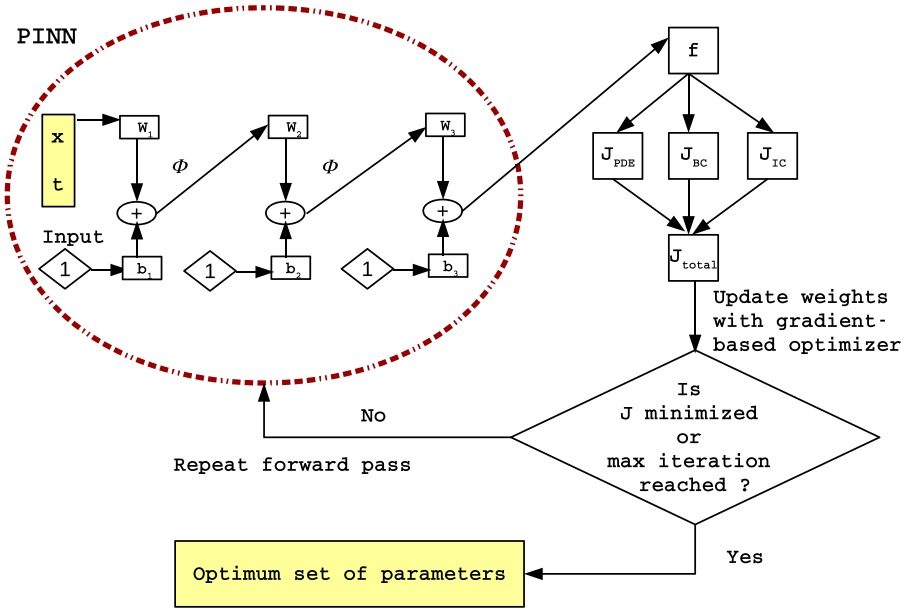


Fig. 3. Schematic diagram of PINN learning procedure.

3.1. Mathematical formulation

Reconsider the 1D unsteady PDE problem given by eqns. (5), (6) and (7). The PIELM for 1D unsteady problem is schematically shown in fig. 4. The number of neurons in the hidden layers is N^*. If we define $\vec{\chi} = [x, t, 1]^T$, $\vec{m} = [m_1, m_2, ..., m_{N^*}]^T$, $\vec{n} = [n_1, n_2, ..., n_{N^*}]^T$, $\vec{b} = [b_1, b_2, ..., b_{N^*}]^T$ and $\vec{c} = [c_1, c_2, ..., c_{N^*}]^T$ then, the output of the k^{th} hidden neuron is

$$h_k = \varphi(z_k),$$

where $z_k = [m_k, n_k, b_k]\vec{\chi}$ and $\varphi = tanh$ is the nonlinear activation function. The PIELM output is given by

$$f(\vec{\chi}) = \vec{h}\,\vec{c}. \tag{12}$$

Given $f(\vec{\chi})$, the exact formulae (without any discretization error) for $\frac{\partial^p f}{\partial x^p}$ and $\frac{\partial f}{\partial t}$ are given by

$$\frac{\partial^p f_k}{\partial x^p} = m_k^p \frac{\partial^p \varphi}{\partial z^p}, \tag{13}$$

$$\frac{\partial f_k}{\partial t} = n_k \frac{\partial \varphi}{\partial z}. \tag{14}$$

The expressions for the errors in approximating the PDE, BCs and IC are same as given in eqns. (8), (9) and (10) respectively. Now, if an ELM with N^* hidden nodes can solve this PDE with zero error, it then implies that there exist $\vec{c}$, $\vec{m}$, $\vec{n}$ and $\vec{b}$ such that

$$\vec{\xi}_f = \vec{0}, \tag{15}$$

$$\vec{\xi}_{bc} = \vec{0}, \tag{16}$$

$$\vec{\xi}_{ic} = \vec{0}. \tag{17}$$

Eqns (15 to 17) can be collectively represented as

$$\boldsymbol{H}\vec{c} = \vec{K}. \tag{18}$$

The form of $\boldsymbol{H}$ and $\vec{K}$ depends on the $\mathcal{N}$, B and F i.e. on the type of PDE, boundary condition and initial condition. In order to find $\vec{c}$, we use the pseudo-inverse as it works well for singular and non square $\boldsymbol{H}$ too. This completes the mathematical formulation of PIELM. The key steps in its implementation are as follows:

1. Assign the input layer weights randomly.

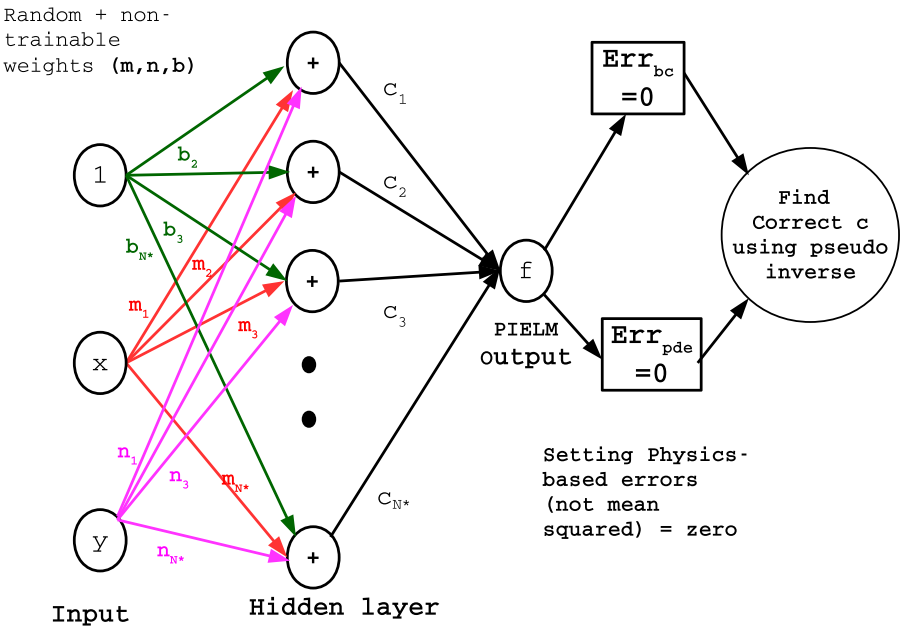


Fig. 4. Schematic diagram of the proposed PIELM for 1D unsteady problems

2. Depending on the PDE and the initial and boundary conditions, find the expressions for $\vec{\xi}_f$, $\vec{\xi}_{bc}$ and $\vec{\xi}_{ic}$.
3. Assemble the three sets of equations in the form of $\boldsymbol{H}\vec{c} = \vec{K}$.
4. Output layer weight vector is given by $pinv(\boldsymbol{H})\vec{K}$, where $pinv$ refers to pseudo-inverse.

3.2. Estimate of architecture

An important advantage of PIELM is that it provides a criterion to decide the number of neurons in the hidden layer. As we saw earlier, if the number of hidden layer nodes is equal to the number of training samples, ELM tends to solve the system of equations exactly. Therefore, instead of choosing the number of hidden units by hit and trial, we set the number of hidden units the same as the number of constraints, i.e., $N^* = N_f + N_{bc} + N_{ic}$. Remarkably our numerical solutions show that the accuracy of prediction is sensitive to the number of neurons for $N < N^*$. For $N > N^*$, the accuracy doesn't improve with an increase in the number of neurons. That is, N^* is the optimal number of neurons for PIELM.

It should be noted that we have not presented any procedure to automatically find the amount of data required to solve any PDE. It is one of the objectives of our future work. For the test cases that will be shown in this paper, we will select the same number of data points as used by the previous authors, and then we will reduce the data gradually. We will also use the method of manufactured solution (MMS) [26] to verify our approach. In MMS, We supply the exact solution to the equation and then calculate the forcing terms and essential boundary conditions using symbolic computation. Therefore, depending upon the exact solution, we will make an educated guess for the size of the dataset.

3.3. PIELM formulation is limited to linear PDEs

Another important point is that the proposed PIELM considers only linear differential operators (with constant or spatially varying coefficients). It is because PIELM solves for the outer layer weights analytically, without using any iterative optimization procedure. For nonlinear operators, we don't get a system of linear equations, and therefore we can not calculate the weights of the outer layer of ELM by any simple matrix inversion. For example, consider the nonlinear inviscid Burgers' equation (refer fig. 4)

$$u_t + uu_x = 0 \tag{19}$$

$$\vec{\xi}_f = \vec{u}_t + \vec{u} \odot \vec{u}_x \tag{20}$$

$$\Rightarrow \xi_f^{(i)} = \varphi'(\vec{x}_i \boldsymbol{W}^T)\vec{c} \odot \vec{n} + \varphi(\vec{x}_i \boldsymbol{W}^T)\vec{c} \odot \varphi'(\vec{x}_i \boldsymbol{W}^T)\vec{c} \odot \vec{m} \tag{21}$$

where $\vec{x}_i = \boldsymbol{X}_f(i,:)$ i.e. i^{th} row of $\boldsymbol{X}_f$ and $\boldsymbol{W} = [\vec{m}, \vec{n}, \vec{b}]$. Eqn (21) can be further simplified to

$$\xi_f^{(i)} = \vec{P}\vec{c} + \vec{c}^T \boldsymbol{Q} \vec{c}$$

where $\vec{P} = diag(\vec{n})\varphi'(\vec{x}_i \boldsymbol{W}^T)^T$ and $\boldsymbol{Q} = \varphi(\vec{x}_i \boldsymbol{W}^T)^T \vec{m}^T diag(\varphi'(\vec{x}_i \boldsymbol{W}^T)^T)$.

In the indicial notation,

$$\xi_f^{(i)} = P_j^{(i)} c_j + c_a Q_{ab}^{(i)} c_b \tag{22}$$

Clearly, this represents a set of nonlinear algebraic equations which can't be solved using a pseudo-inverse operation. Therefore, we are solving only linear PDEs in this paper.

4. Performance evaluation of PIELM

To evaluate the performance of PIELM, we rigorously test it on various linear and quasi-linear PDEs described in Table (1). TC-1, TC-2, TC-4, TC-5, TC-6 are taken from Berg et al. [7]. TC-8 is taken from Kopriva et al. [27]. All the experiments are conducted in Matlab 2017b environment running in an Intel Core i5 2.20GHz CPU and 8GB RAM Dell laptop. The error is defined as the difference between the PIELM prediction and the exact solution.

4.1. 1D steady cases [TC-1, TC-2, TC-3]

The 1D stationary advection, diffusion and advection-diffusion equations are given by

$$u_x = R, 0 < x \leq 1, \tag{23}$$

Table 1
List of test cases for PIELM.

Steady/ Unsteady	1D/2D	Test case ID	Description
Steady	1D	TC-1	Linear advection
		TC-2	Linear diffusion
		TC-3	Linear advection-diffusion
	2D	TC-4	Linear advection in a star shaped computational domain
		TC-5	Linear diffusion in a star shaped computational domain
		TC-6	Linear diffusion in a complex computational domain
Unsteady	1D	TC-7	Linear advection (constant coefficient)
		TC-8	Linear advection (spatially variable coefficient)

Table 2
Summary of experiments for 1D steady test cases.

TC	$[N_f, N_{bc}, N^*]$	$\mathcal{O}(Error)$	$\mathcal{O}(t_{CPU})$
TC-1	[100,2,102]	10^{-3}	10^{-3} seconds
TC-2	[100,2,102]	10^{-3}	10^{-3} seconds
TC-3	[20,2,22]	10^{-6}	10^{-3} seconds

$$u_{xx} = R, 0 < x \leq 1, \tag{24}$$

$$u_x - \nu u_{xx} = R, 0 < x < 1 \tag{25}$$

respectively. The expressions for R and the Dirichlet boundary conditions for these cases are calculated by assuming the following exact solutions

$$\widehat{u} = sin(2\pi x)cos(4\pi x) + 1, \tag{26}$$

$$\widehat{u} = sin(\frac{\pi x}{2})cos(2\pi x) + 1, \tag{27}$$

$$\widehat{u} = \frac{e^{\frac{x}{\nu}} - 1}{e^{\frac{1}{\nu}} - 1} \tag{28}$$

respectively. In order to solve these equations in PIELM framework, we have to solve eqn(15 to 17). The expression for $\vec{\xi}_f$ depends on the linear differential operator $\mathcal{N}$. The definitions of $\mathcal{N}$ in these cases are $\frac{\partial}{\partial x}$, $\frac{\partial^2}{\partial x^2}$ and $\frac{\partial}{\partial x} - \nu\frac{\partial^2}{\partial x^2}$ respectively. When $\mathcal{L}$ acts on u, the corresponding expressions for $\vec{\xi}_f = \vec{0}$ may be written as follows:

$$\varphi'(\boldsymbol{X_f W}^T)\odot\vec{c}\odot\vec{m} = R(\vec{x}_f), \tag{29}$$

$$\varphi''(\boldsymbol{X_f W}^T)\odot\vec{c}\odot\vec{m}\odot\vec{m} = R(\vec{x}_f), \tag{30}$$

$$\varphi'(\boldsymbol{X_f W}^T)\odot\vec{c}\odot\vec{m} - \nu\varphi''(\boldsymbol{X_f W}^T)\odot\vec{c}\odot\vec{m}\odot\vec{m} = \vec{0}. \tag{31}$$

where $\vec{x}_f$ is collocation points vector, $\vec{1}$ is bias vector, $\boldsymbol{X_f} = [\vec{x}_f, \vec{1}]$ and '$\odot$' refers to Hadamard product. Referring to fig. 5, $\boldsymbol{W} = [\vec{m}, \vec{b}]$. Similarly, expression for $\vec{\xi}_{bc} = \vec{0}$ is given by

$$\varphi(\boldsymbol{X_{bc} W}^T)\vec{c} = B(\vec{x}_{bc}) \tag{32}$$

where $\vec{x}_{bc}$ is boundary points vector, $\boldsymbol{X_{bc}} = [\vec{x}_{bc}, \vec{1}]$ and B is the boundary condition.

The results for these test cases are given in figs. 6, 7, 8 and 14 respectively and the summary of the experiments is given in Table (2).

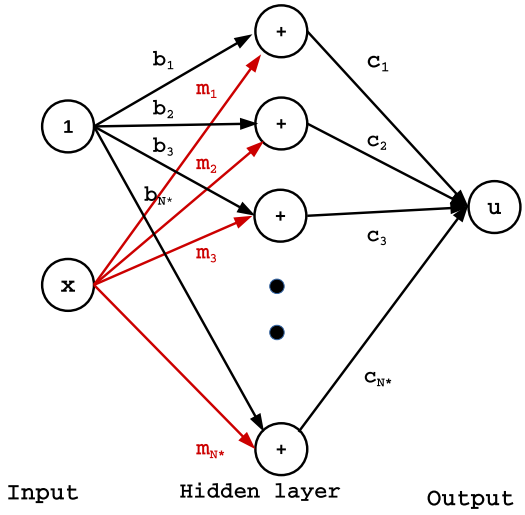


Fig. 5. PIELM for steady 1D problem

4.2. 2D steady cases [TC-4, TC-5, TC-6]

The stationary 2D advection and diffusion equations for the three cases are given by

$$au_x + bu_y = R, (x, y) \in \Omega_1 \tag{33}$$

$$u_{xx} + u_{yy} = R, (x, y) \in \Omega_1 \tag{34}$$

$$u_{xx} + u_{yy} = R, (x, y) \in \Omega_2 \tag{35}$$

where $a = 1$ and $b = \frac{1}{2}$ are advection coefficients. The computational domains Ω_1 and Ω_2 are shown in respectively. The expressions for R and the B for these cases are constructed by choosing the following exact solutions

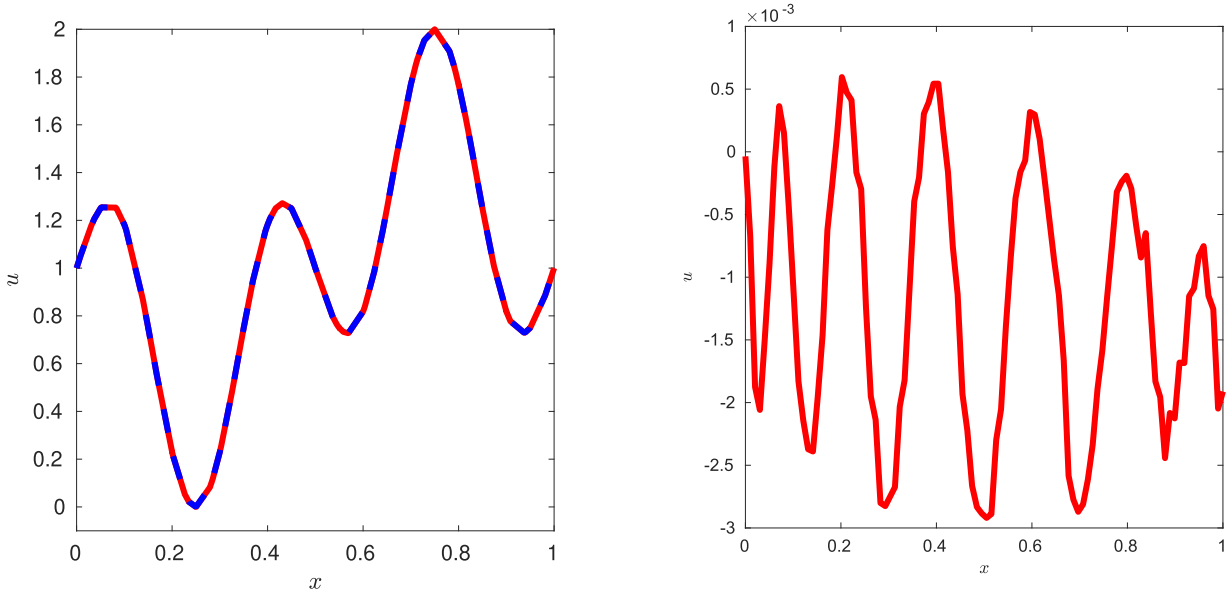
$$\widehat{u} = \frac{1}{2}cos(\pi x)sin(\pi y), \tag{36}$$

$$\widehat{u} = \frac{1}{2} + e^{-(2x^2+4y^2)}, \tag{37}$$

$$\widehat{u} = \frac{1}{2} + e^{-((x-0.6)^2+(y-0.6)^2)} \tag{38}$$

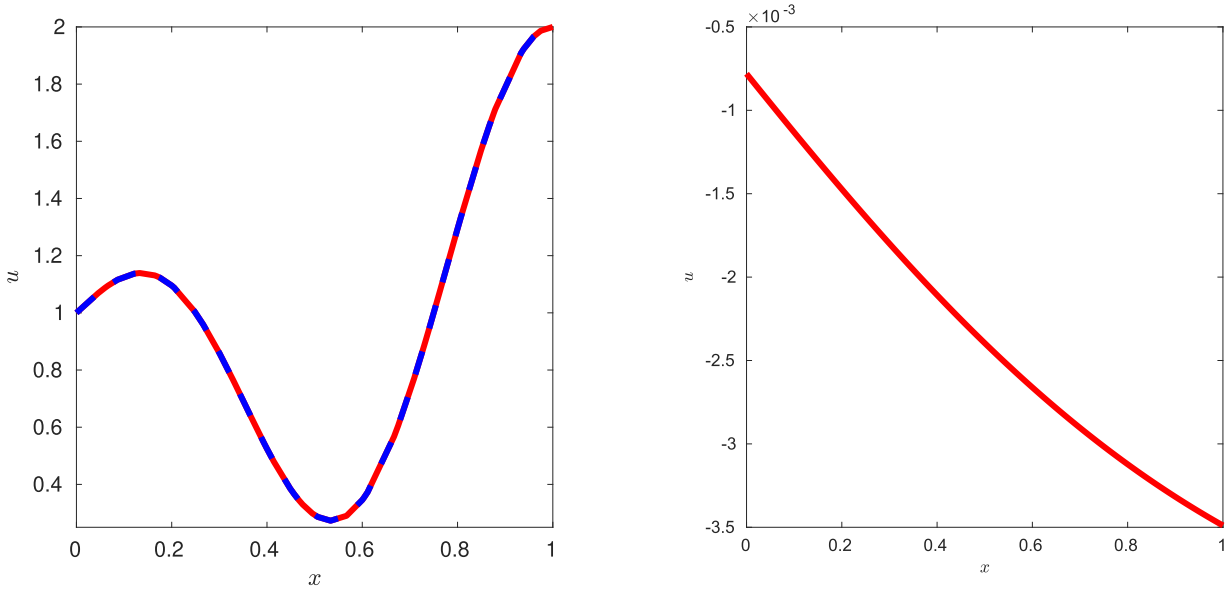
respectively. PIELM equations for these problems are as follows:

1. $\vec{\xi}_f = \vec{0}$
   - Case 1: Advection
     $$\varphi'(\boldsymbol{X_f W}^T)\odot\vec{c}\odot(a\vec{m} + b\vec{n}) = R(\vec{x}_f, \vec{y}_f) \tag{39}$$
   - Case 2: Diffusion
     $$\varphi''(\boldsymbol{X_f W}^T)\odot\vec{c}\odot(\vec{m}\odot\vec{m} + \vec{n}\odot\vec{n}) = R(\vec{x}_f, \vec{y}_f) \tag{40}$$
     where $\boldsymbol{X_f} = [\vec{x}_f, \vec{y}_f, \vec{1}]$ and $\boldsymbol{W} = [\vec{m}, \vec{n}, \vec{b}]$ (refer fig. 4).



(a) PIELM solution. (b) Error plot.

Fig. 6. Solution and error plots for steady 1D advection. Red: PIELM, Blue: Exact.



(a) PIELM solution. (b) Error plot.

Fig. 7. Solution and error plots for steady 1D diffusion. Red: PIELM, Blue: Exact.

2. $\vec{\xi}_{bc} = \vec{0}$

$$\varphi(\boldsymbol{X_{bc}}\boldsymbol{W}^T)\vec{c} = B(\vec{x}_{bc}, \vec{y}_{bc}) \tag{41}$$

where $\boldsymbol{X_{bc}} = [\vec{x}_{bc}, \vec{y}_{bc}, \vec{1}]$.

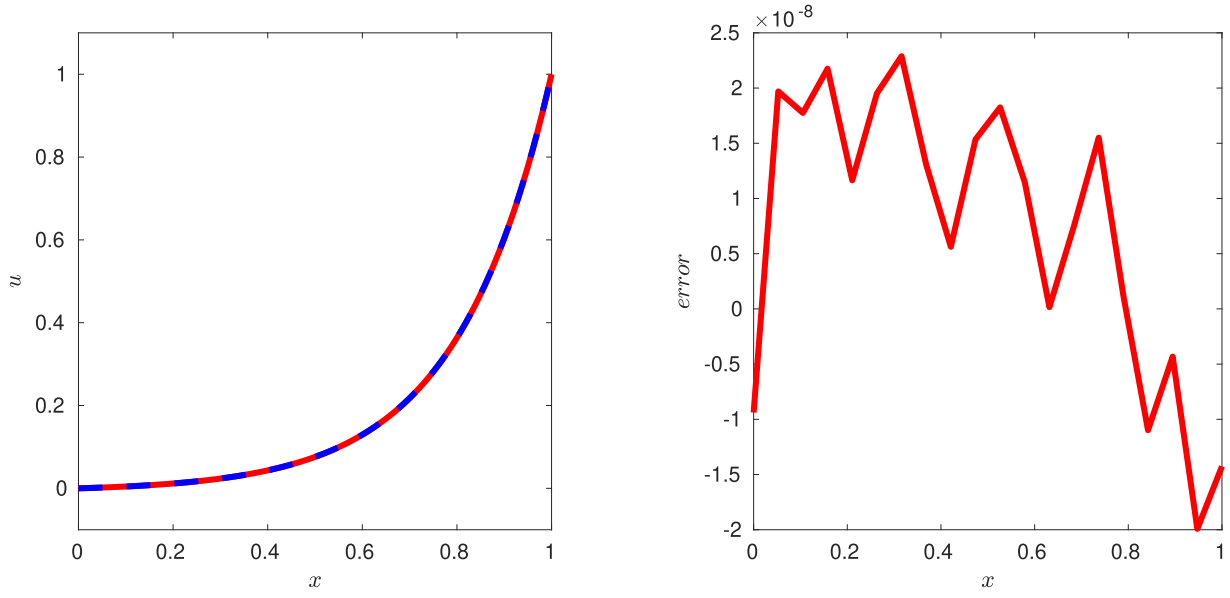
The results for the advection and diffusion cases on Ω_1 are shown in fig. 10 and fig. 11 respectively. The result for diffusion case on Ω_2 is given in fig. 12. Summary of the experiments is given in Table (3). A tabular comparison between unified deep ANN [7] and PIELM is given in Table (4)

Remarks

1. The unified deep ANN algorithm [7] took 5500 points to achieve an order of accuracy of 10^{-3} in steady 2D diffusion for star-shaped geometry. In comparison, PIELM solved these cases with merely 1161 points and still achieved an order of accuracy of 10^{-6} and 10^{-4}, respectively.
2. False diffusion is an error that gives diffusion like appearance to the solution of pure advection equation when it is solved using upwind discretization. In fig. 10 b these errors can be seen flowing along streamlines. However, PIELM reduces the order of these errors to an insignificant level. It is consistent with our

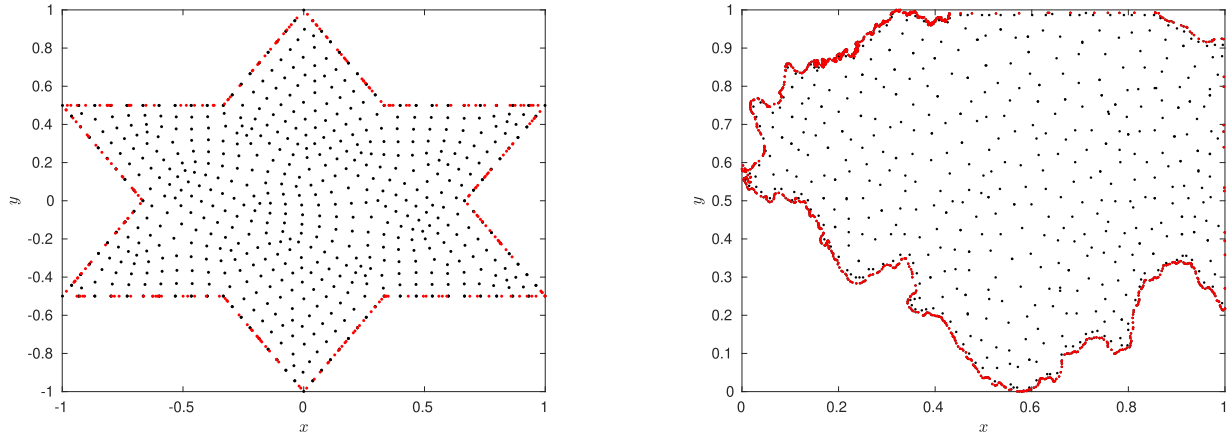
Table 3
Summary of experiments for 2D steady test cases.

TC	$[N_f, N_{BC}, N^*]$	$\mathcal{O}(Error)$	t_{CPU}
TC-4	[921,240,2000]	10^{-6}	∼1 second
TC-5	[921,240,2000]	10^{-4}	∼1 second
TC-6	[1489,881,5000]	10^{-7}	∼10 seconds



(a) PIELM solution. (b) Error plot.

Fig. 8. Solution and error plots for steady 1D advection diffusion at $\nu = 0.2$. Red: PIELM, Blue: Exact.

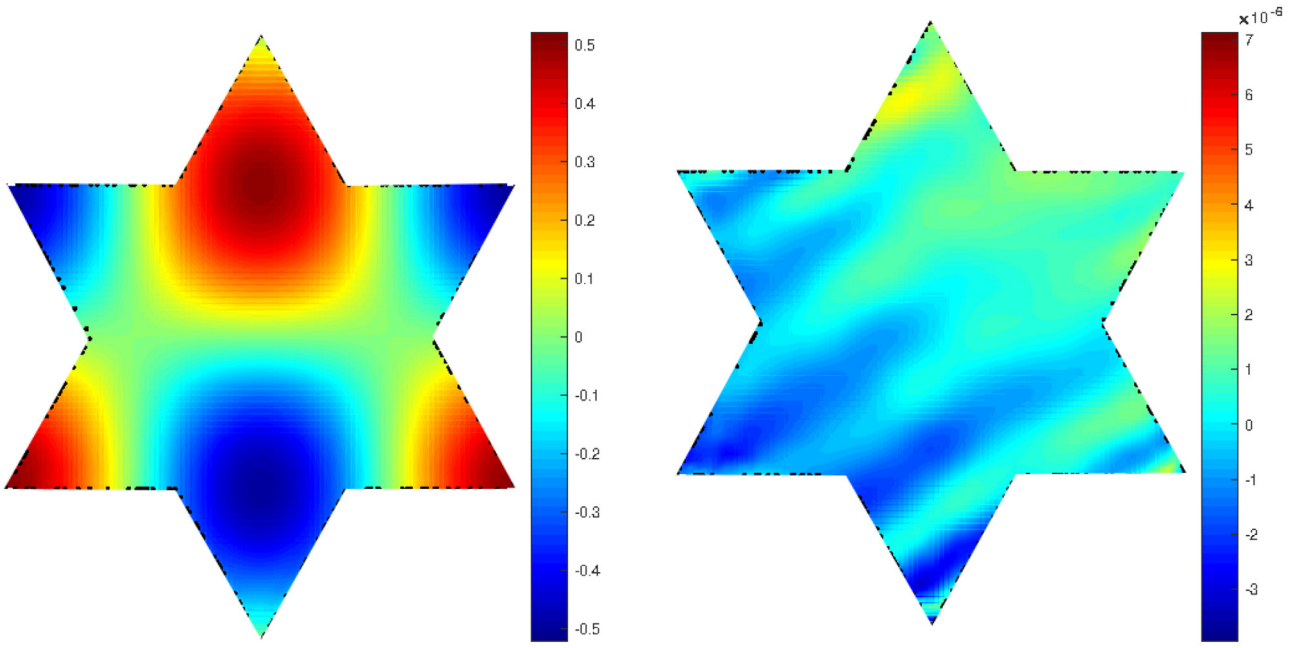


(a) Ω_1 (b) Ω_2

Fig. 9. Sampling points for 2D steady problems

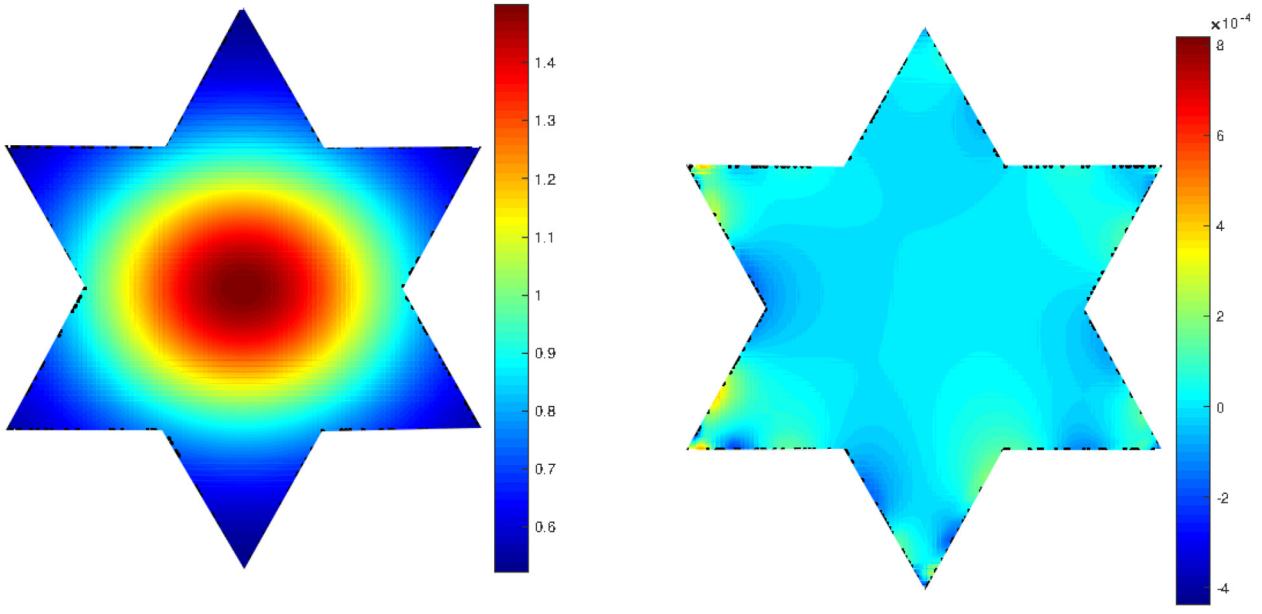
Table 4
Comparison between unified deep ANN and PIELM

	Unified deep ANN	PIELM
Target PDE	Stationary PDE with Dirichlet BCs.	Stationary and time dependent linear PDE with no restriction on BCs.
Ansatz	$\widehat{u}(x) = G(x) + D(x)y(x; w, b)$ (BCs are inherently satisfied)	$\widehat{u}(x) = y(x; w, b)$ (BCs are not satisfied inherently)
Architecture	• Two shallow neural networks for boundary data extension and smooth distance function, One deep neural network for collocation data. • All parameters are trainable.	• Single shallow neural network for both collocation and boundary data. • Only output layer weights are trainable.
Loss function	Physics based MSE.	Physics based MSE.
Performance (2D diffusion equation)	• Data set: $[N_f, N_{bc}] = [5000, 500]$ • CPU time: ~ 8 hrs. • Accuracy: 10^{-5}	• Data set: $[N_f, N_{bc}] = [921, 240]$ • CPU time: few seconds. • Accuracy: 10^{-4}
Limitations	• Time consuming. • Vanishing gradient problems for deep networks.	• Valid for linear PDEs only.



(a) PIELM solution. (b) Error plot.

Fig. 10. Solution and error for 2D steady advection equation on Ω_1.



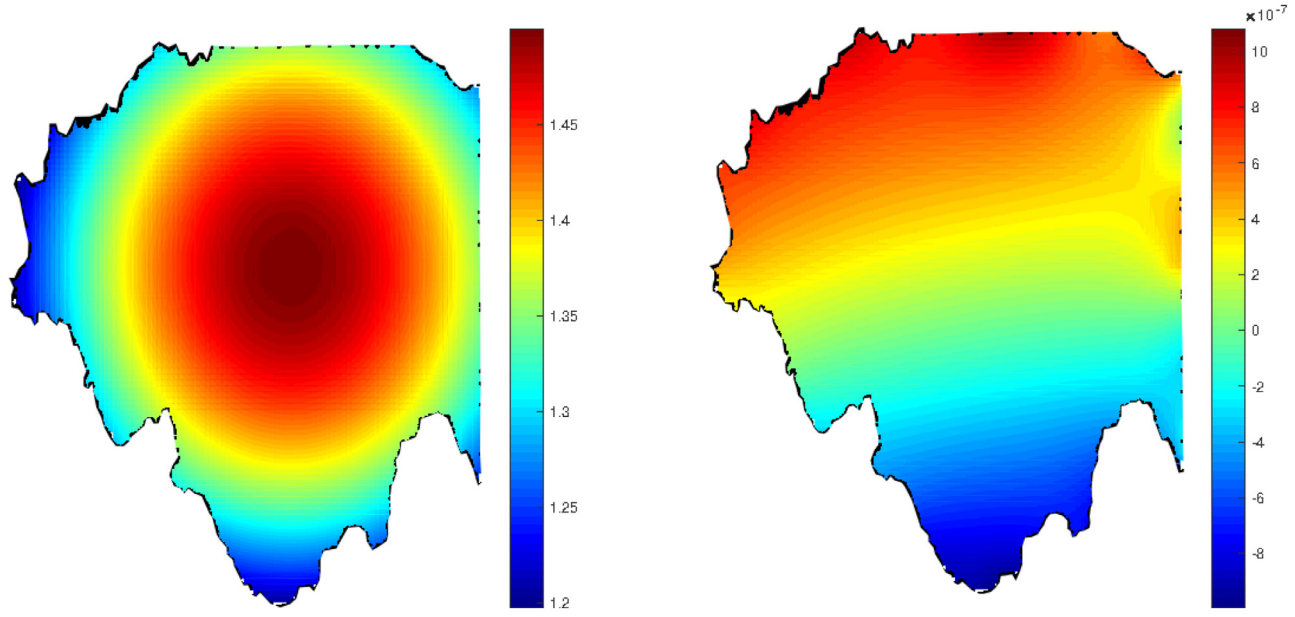
(a) PIELM solution. (b) Error plot.

Fig. 11. Solution and error for 2D steady diffusion equation on Ω_1.

assumption that better PDE representation in basis leads to a more robust method.

3. Computational domain Ω_2 is the map of the state of Illinois in the USA. It is an example of a complicated polygon which has very short line segments and fine-grained details in various regions. Conventional mesh-based methods are not feasible for these kinds of geometries. The latitudes and longitudes of the boundary are available in MATLAB's in-built function "usamap". We have re-scaled the data in the range of $0-1$. PIELM solved this case with just 2370 points to an accuracy level of 10^{-7}.
4. To solve 2D diffusion problem, deep unified ANN takes nearly 35000 iterations for the solution to reach an accuracy of 10^{-5}. The details of the network are as follows:
   - Architecture: 5 hidden layers with 20 neurons in each layer.
   - Dataset: $N_f = 5000$, $N_{bc} = 500$.

   In the unified deep-ANN, a second order (L-BFGS) optimizer was used. However, even for a first order optimizer like Adam



(a) PIELM solution. (b) Error plot.

Fig. 12. Solution and error for 2D steady diffusion equation on Ω_2.

optimizer, one iteration requires nearly 0.83 seconds. So the estimated total CPU time for solving 2D diffusion equation is almost 8 hours! On the same computer, PIELM takes only a few seconds to solve this PDE with much less input data $[N_f, N_{bc}] = [921, 240]$.

4.3. 1D unsteady advection cases [TC-7, TC-8]

The unsteady 1D advection equation is given by

$$u_t + a(x)u_x = 0, (x,t) \in (-1,1) \times (0,0.5), \tag{42}$$

$$u(x,0) = F(x), x \in (-1,1), \tag{43}$$

where $a(x, t)$ is the advection coefficient. In this problem, we consider two cases: (1) constant coefficient case with $a(x,t) = 1$ and (2) variable coefficient case [27] with $a(x,t) = 1 + x$. The two cases have periodic and inflow boundary conditions, respectively. The value of F is $sin(\pi x)$. The exact solutions to the 1D unsteady advection problems with constant and variable coefficient are respectively given by

$$\widehat{u} = F(x - t), \tag{44}$$

$$\widehat{u} = F((1+x)e^{-t} - 1). \tag{45}$$

PIELM equations

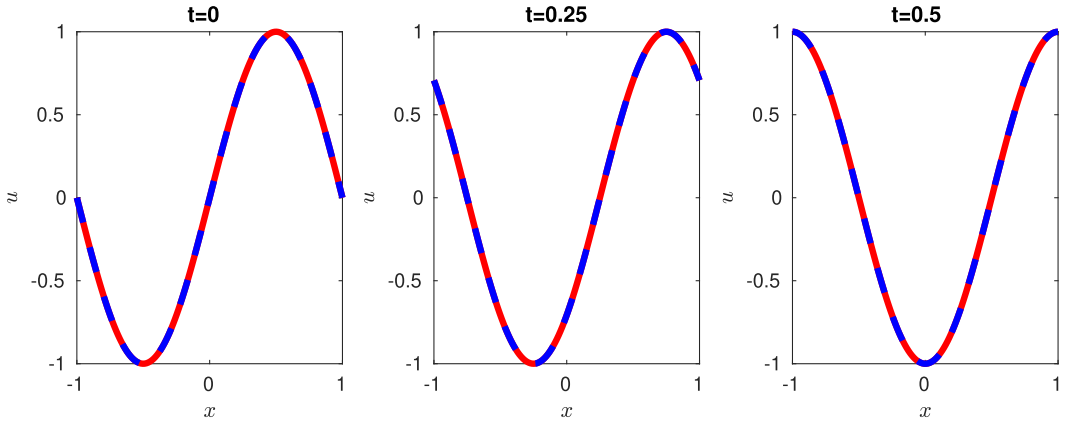
1. $\vec{\xi}_f = \vec{0}$
   - TC-7: Constant coefficient
     $$\varphi'(\boldsymbol{X_f W}^T) \odot \vec{c} \odot (\vec{m} + \vec{n}) = \vec{0} \tag{46}$$
   - TC-8: Spatially varying coefficient
     $$\varphi'(\boldsymbol{X_f W}^T)\vec{c} \odot \vec{n} + (\varphi'(\boldsymbol{X_f W}^T) \odot \boldsymbol{A_f})\vec{c} \odot \vec{m} = \vec{0} \tag{47}$$
     where $\vec{x}_f, \vec{t}_f$ are collocation point vectors, $\boldsymbol{X_f} = [\vec{x}_f, \vec{t}_f, \vec{1}]$, $\boldsymbol{W} = [\vec{m}, \vec{n}, \vec{b}]$ and $\boldsymbol{A_f} = [\ a(\vec{x_f}, \vec{t_f}), \ \ldots \ , \ a(\vec{x_f}, \vec{t_f})\ ]_{N \times N^*}$.
2. $\vec{\xi}_{bc} = \vec{0}$
   $$\varphi(\boldsymbol{X_{lbc} W}^T)\vec{c} = \varphi(\boldsymbol{X_{rbc} W}^T)\vec{c} \tag{48}$$
   where $\vec{x}_{lbc}$, $\vec{t}_{lbc}$ are left boundary points vectors, $\vec{x}_{rbc}$, $\vec{t}_{rbc}$ are right boundary points vectors, $\boldsymbol{X_{lbc}} = [\vec{x}_{lbc}, \vec{t}_{lbc}, \vec{1}]$ and $\boldsymbol{X_{rbc}} = [\vec{x}_{rbc}, \vec{t}_{rbc}, \vec{1}]$.
3. $\vec{\xi}_{ic} = \vec{0}$
   $$\varphi(\boldsymbol{X_{ic} W}^T)\vec{c} = F(\vec{x}_{ic}, \vec{t}_{ic}) \tag{49}$$
   where $\vec{x}_{ic}, \vec{t}_{ic}$ are initial condition vectors, $\boldsymbol{X_{ic}} = [\vec{x}_{ic}, \vec{t}_{ic}, \vec{1}]$ and F is initial condition.

The results are shown in fig. 13. PIELM predicts the exact solution correctly for both linear and quasi-linear advection. In this case, we took $N_f = 420$, $N_{bc} = 21$, $N_{ic} = 20$ and $N^* = 440$. The total learning time is within 2-3 seconds.

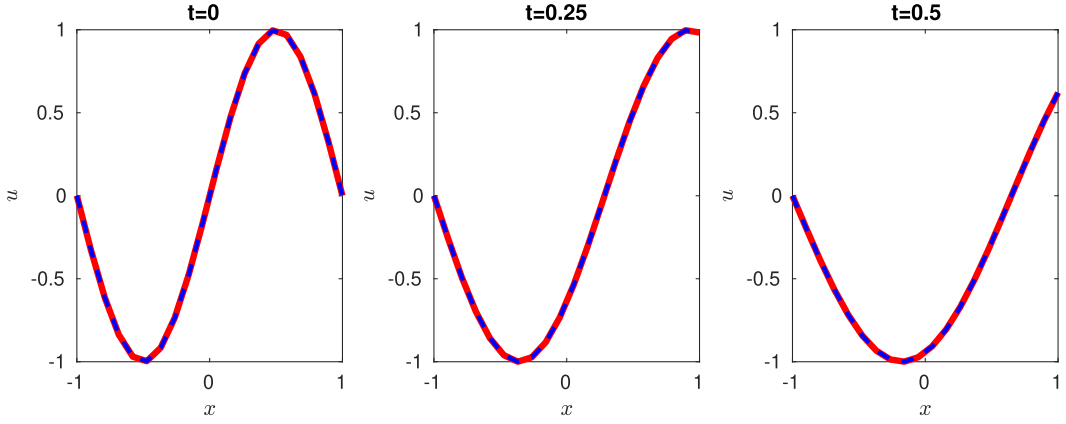
For unsteady PDEs, conventional methods put a limitation on time step depending upon mesh size. For example, the upwind differencing scheme gives correct results for the advection equation (TC-7) only when time step size is the same as mesh size (the famous Courant - Friedrichs - Lewy condition [28]). If the time step is less than the mesh size, the solution shows dissipation errors. If the time step is more than the mesh size, the solution blows up due to numerical instability. PIELM, on the other hand, doesn't suffer from such issues.

4.4. Effect of number of hidden neurons on PIELM solutions

For a dataset consisting of 100 points i.e., $N = 100$, we changed the size of the hidden layer for different steady 1D cases (TC-1 to TC-3). The result for a 1D steady advection case has been shown in fig. 14 for reference. It can be observed that PIELM gives the best results when the size of the hidden layer is the same as the number of equations. However, it can infer the solutions even with a lesser number of neurons. Increasing the number of neurons beyond the number of equations leads to an initial drop in accuracy followed by saturation to an even more inaccurate solution.



(a) Constant coefficient unsteady advection.



(b) Variable coefficient unsteady advection.

Fig. 13. Exact and PIELM solution for 1D unsteady advection with constant and variable coefficients. Red: PIELM, Blue: Exact.

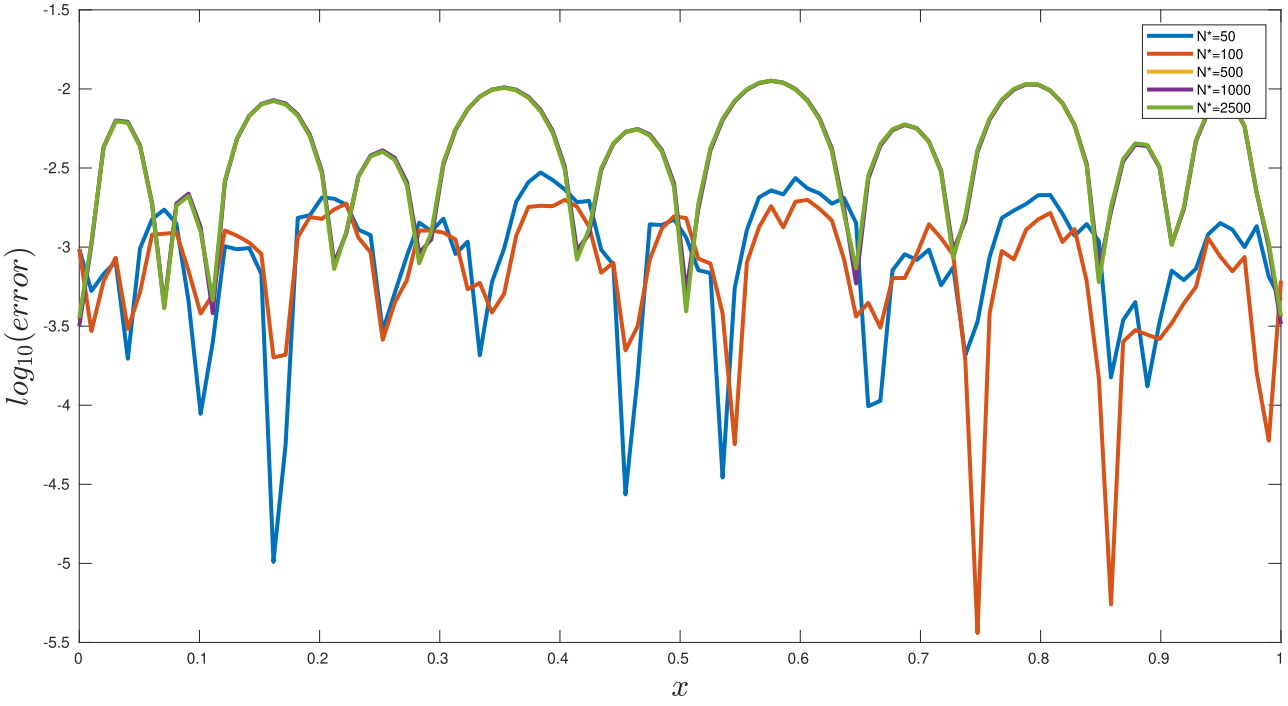


Fig. 14. Effect of number of hidden nodes on PIELM solution for steady advection case (N=100).

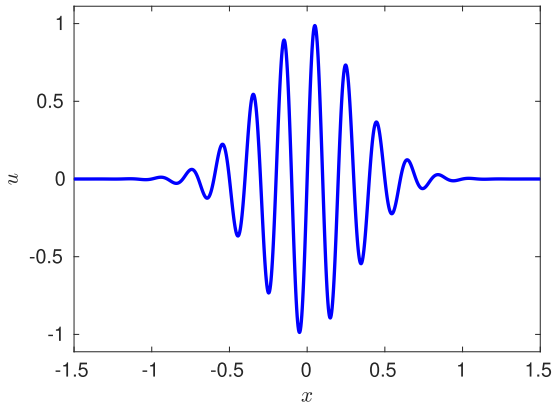


Fig. 15. Initial condition with sharp gradients.

4.5. Summary

In summary, the highlights of PIELM described so far are as follows:

1. It can solve steady and time-dependent linear PDEs.
2. It is extremely fast as well as data efficient. (TC-4 to TC-6)
3. It reduces numerical artifacts like false diffusion. (TC-4)
4. It is a meshfree method and can handle complex geometries. (TC-6)
5. It reduces the arbitrariness in the number of neurons in the hidden layer. (TC 1-8)

5. Limitations of PIELM

In the context of binary classification problem, it is a well-known fact that the learning algorithm may suffer from *overfitting* if a complex-shaped boundary partitions the two groups of data [29]. Generally, regularization is used to counter this issue. In the context of the solution of PDEs with PINN, the physics-based loss function acts as a regularizer. However, if the PDE admits complex-shaped solutions, it is indeed a hard task for a single neural network to approximate the solution properly.

In this Section, firstly, we will present an interesting case of failure of PINN in solving the linear advection equation subjected to a complicated initial condition. Next, we will show that our PIELM also exhibits this behavior for a series of test cases involving the representation of intricate functions and solutions of PDEs, which allow sharp gradient solutions. It is to be noted that an increase in the number of data points or number of layers is not a solution because the location of sharp gradients is unknown to us apriori and any increase in data or hidden layers will increase the computational cost of the solution without any guarantee of convergence.

5.1. Failure of deep PINN in solving linear advection equation with complicated initial condition

Consider the linear advection equation (TC-7). We set $F = e^{-x^2} sin(10\pi x)$ as the initial condition. The plot of the initial condition is given in fig. 15. To solve this equation with a deep PINN, we download the original PINN code for nonlinear Burgers' equation from https://github.com/maziarraissi/PINNs and simplify it for the linear advection case. The deep PINN consists of 9 hidden layers with 20 neurons each. The exact and the PINN solutions are shown in fig. 16. We can see that even a deep network is unable to capture the sharp gradients of this function.

5.2. Failure of PIELM in solving PDEs with sharp gradient solutions

In the rest of this section, we will expose the weaknesses of PIELM in representing complicated functions and solving PDEs with sharp gradient solutions. The list of the selected test cases is given in the Table (5) given below.

5.2.1. Representation of functions with sharp gradients [TC-9, TC-10]

To put the representation power of PIELM to test we choose the following two functions: (1) A 1D composite function that contains both discontinuous functions and functions with sharp gradients and (2) A 2D Gaussian function with a sharp peak. The expressions for 2D Gaussian function and 1D composite function are $f(x, y) = e^{-20\{x^2+y^2\}}$ and

$$f(x) = \begin{cases} \frac{1}{2}\{sgn(x+0.8) - sgn(x+0.5)\} & x \le -\frac{1}{2} \\ e^{-100x^2} & -\frac{1}{2} < x \le \frac{1}{2} \\ \frac{20}{3}x - \frac{10}{3} & \frac{1}{2} < x \le \frac{13}{20} \\ -\frac{20}{3}x + \frac{16}{3} & \frac{13}{20} < x \le \frac{4}{5} \\ 0 & x > \frac{4}{5} \end{cases} \tag{50}$$

respectively. The results are shown in fig. 17 and fig. 18 respectively. These cases clearly expose the limitation of PIELM in representing profiles with sharp gradients and corners.

5.2.2. PDEs with sharp solutions [TC-11, TC-12]

PIELM fails to solve any PDE which admits functions with sharp gradients. For example, we have already seen the failure of our algorithm in solving 1D and 2D advection-diffusion equation. We further illustrate the impact of this limitation on two simpler equations. Firstly, we consider the pure advection of a sharp-peaked Gaussian. This equation has been solved in TC-7 for a smooth sine function. Next, we take a steady advection-diffusion equation (TC-3) with a low value of the diffusion coefficient. The exact and PIELM solutions for these problems are shown in fig. 19 and fig. 20 respectively.

5.2.3. 1D unsteady advection-diffusion [TC-13]

The 1D equivalent of the unsteady 2D advection-diffusion equation solved by Borker et al. [30] is given by

$$u_t + au_x = \nu u_{xx}, (x, t) \in (0, 1)\text{x}(0, 0.5), \tag{51}$$

where a is the advection coefficient. The expressions for the initial condition F and the boundary condition B are constructed on the basis of the following exact solution

$$\widehat{u} = \frac{1}{\sqrt{4t+1}} e^{-\frac{1}{\nu(4t+1)}(x-0.2-at)^2} \tag{52}$$

The results are shown in fig. 21. The PIELM equations are as follows:

1. $\vec{\xi}_f = \vec{0}$

$$\varphi'(\boldsymbol{X_f W}^T)\vec{c} \odot \vec{n} + a\varphi'(\boldsymbol{X_f W}^T)\vec{c} \odot \vec{m} = \nu\varphi''(\boldsymbol{X_f W}^T) \odot \vec{c} \odot \vec{m} \odot \vec{m} \tag{53}$$

where $\vec{x}_f$, $\vec{t}_f$ are collocation point vectors, $\boldsymbol{X_f} = [\vec{x}_f, \vec{t}_f, \vec{1}]$ and $\boldsymbol{W} = [\vec{m}, \vec{n}, \vec{b}]$.

2. $\vec{\xi}_{bc} = \vec{0}$

$$\varphi(\boldsymbol{X_{bc} W}^T)\vec{c} = B(\vec{x}_{bc}, \vec{t}_{bc}) \tag{54}$$

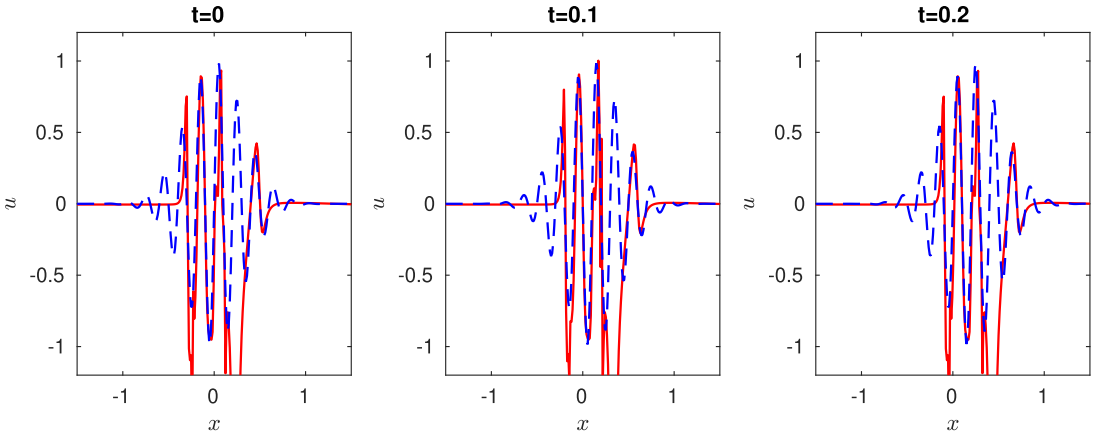


Fig. 16. Exact and PINN solution of pure advection of a high frequency wave packet. Red: PINN, Blue: exact.

Table 5
List of test cases where PIELM fails.

Test case ID	Description	$[N_f, N_{BC}, N^*]$
TC-9	Representation of 1D sharp and discontinuous functions	[500,2,502]
TC-10	Representation of a sharp peaked 2D Gaussian	[2500,200,2700]
TC-11	1D steady advection-diffusion	[500,2,502]
TC-12	Advection of a sharp peaked Gaussian	[2500,200,2700]
TC-13	1D unsteady linear advection-diffusion	[1250,150,1400]
TC-14	2D unsteady linear advection-diffusion equation	[4000,2000,6000]

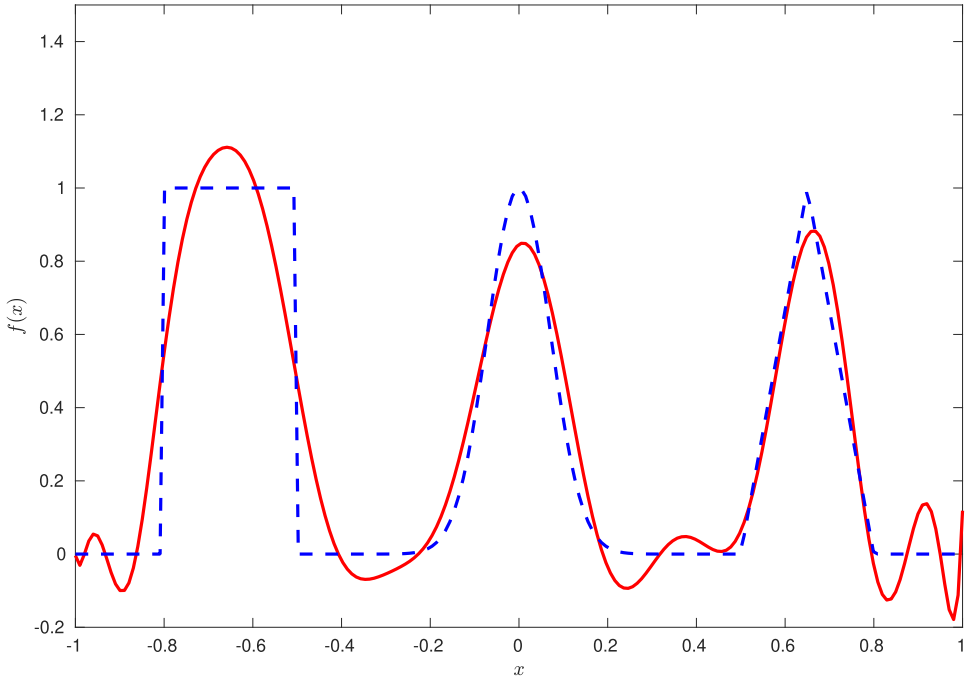


Fig. 17. Representation of 1D non smooth functions with PIELM. Red: PIELM solution, Blue: exact solution.

where $\vec{x}_{bc}$, $\vec{t}_{bc}$ are boundary points vectors, $\boldsymbol{X_{bc}} = [\vec{x}_{bc}, \vec{t}_{bc}, \vec{1}]$ and B is the boundary condition.

3. $\vec{\xi}_{ic} = \vec{0}$

$$\varphi(\boldsymbol{X_{ic}}\boldsymbol{W}^T)\vec{c} = F(\vec{x}_{ic}, \vec{t}_{ic}) \tag{55}$$

where $\vec{x}_{ic}$, $\vec{t}_{ic}$ are initial condition vectors, $\boldsymbol{X_{ic}} = [\vec{x}_{ic}, \vec{t}_{ic}, \vec{1}]$ and F is initial condition.

5.2.4. 2D unsteady advection-diffusion [TC-14]

We solve the following unsteady 2D advection-diffusion equation [30]

$$u_t + au_x + bu_y = \nu(u_{xx} + u_{yy}),\ (x, y, t) \in \Omega_{xy} \times (0, 0.2), \tag{56}$$

$$u(x, y, 0) = F(x, y),\ (x, y) \in \Omega_{xy}, \tag{57}$$

$$u(x, y, t) = B(x, y, t),\ (x, y, t) \in \partial\Omega_{xy} \times (0, 0.5), \tag{58}$$

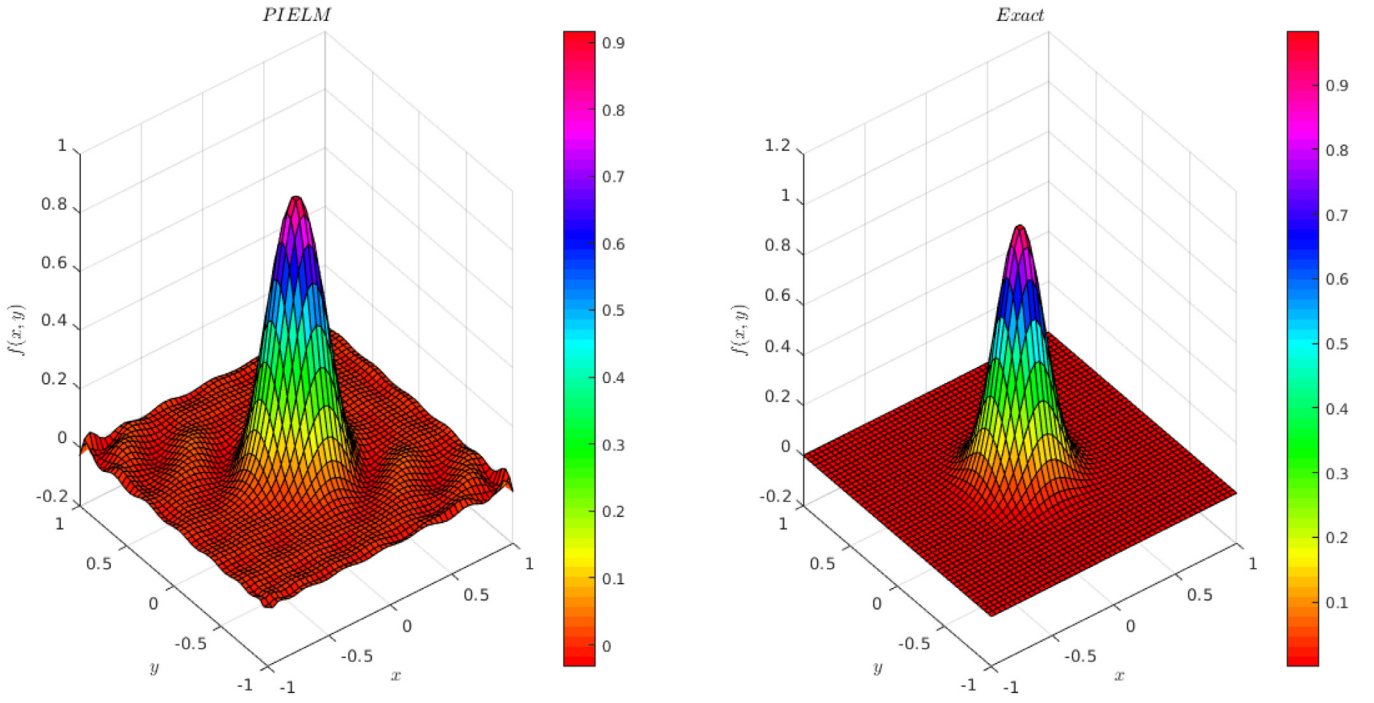


Fig. 18. Representation of a sharp peaked 2D Gaussian with PIELM

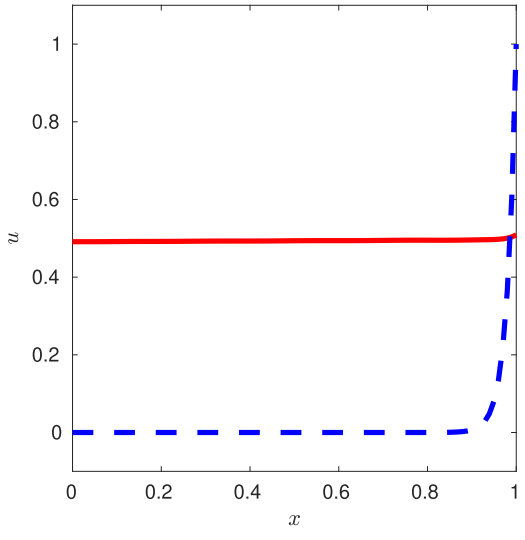


Fig. 19. Steady 1D convection diffusion at $\nu = 0.02$. Red: PIELM, Blue: Exact.

where $\Omega_{xy} = [0, 1]$x$[0, 1]$. $a = cos(22.5)$ and $b = sin(22.5)$ are advection coefficients in x and y directions and $\nu = 0.005$ is the diffusion coefficient. The expressions for the initial and the boundary conditions are constructed by choosing the following exact solution:

$$\widehat{u} = \frac{1}{(4t+1)} e^{-\frac{(x-at)^2+(y-bt)^2}{\nu(4t+1)}} \tag{59}$$

The results are shown in figs. 22-23. The PIELM equations to be solved are as follows:

1. $\overrightarrow{\xi}_f = \overrightarrow{0}$

$$\varphi'(\boldsymbol{X_f W}^T)\overrightarrow{c} \odot \overrightarrow{r} + \varphi'(\boldsymbol{X_f W}^T)\overrightarrow{c} \odot (a\overrightarrow{p} + b\overrightarrow{q}) = \nu\varphi''(\boldsymbol{X_f W}^T) \odot \overrightarrow{c} \odot (\overrightarrow{p} \odot \overrightarrow{p} + \overrightarrow{q} \odot \overrightarrow{q}) \tag{60}$$

where $\overrightarrow{x}_f, \overrightarrow{y}_f, \overrightarrow{t}_f$ are collocation point vectors, $\boldsymbol{X_f} = [\overrightarrow{x}_f, \overrightarrow{y}_f, \overrightarrow{t}_f, \overrightarrow{1}]$, $\boldsymbol{W} = [\overrightarrow{p}, \overrightarrow{q}, \overrightarrow{r}, \overrightarrow{s}]$.

2. $\overrightarrow{\xi}_{bc} = \overrightarrow{0}$

$$\varphi(\boldsymbol{X_{bc} W}^T)\overrightarrow{c} = B(\overrightarrow{x}_{bc}, \overrightarrow{y}_{bc}, \overrightarrow{t}_{bc}) \tag{61}$$

where $\overrightarrow{x}_{bc}, \overrightarrow{y}_{bc}, \overrightarrow{t}_{bc}$ are boundary points vectors and $\boldsymbol{X_{bc}} = [\overrightarrow{x}_{bc}, \overrightarrow{y}_{bc}, \overrightarrow{t}_{bc}, \overrightarrow{1}]$.

3. $\overrightarrow{\xi}_{ic} = \overrightarrow{0}$

$$\varphi(\boldsymbol{X_{ic} W}^T)\overrightarrow{c} = F(\overrightarrow{x}_{ic}, \overrightarrow{y}_{ic}, \overrightarrow{t}_{ic}) \tag{62}$$

where $\overrightarrow{x}_{ic}, \overrightarrow{y}_{ic}, \overrightarrow{t}_{ic}$ are initial condition vectors and $\boldsymbol{X_{ic}} = [\overrightarrow{x}_{ic}, \overrightarrow{y}_{ic}, \overrightarrow{t}_{ic}, \overrightarrow{1}]$.

6. Distributed PIELM

In this section, we develop distributed physics informed extreme learning machines. Traditional FEM inspires the idea of DPIELM. DPIELM involve the division of actual computational domains into multiple sub-domains. The function (and thus PDE) in each sub-domain is represented by simple neural networks. These representations are stitched together by interface constraints of smoothness and curvature. Typically, the use of simple polynomials as basis functions enhances the representation capacity of FEM. In our context, we show that the use of piecewise neural networks enhances the representation capacity of PIELM.

6.1. Problem Definition

Consider the following 1D unsteady problem

$$\frac{\partial}{\partial t} u(x,t) + \mathcal{N}u(x,t) = R(x,t), (x,t) \in \Omega \tag{63}$$

$$u(x,t) = B(x,t), (x,t) \in \partial\Omega, \tag{64}$$

$$u(x,0) = F(x), x \in (x_L, x_R), \tag{65}$$

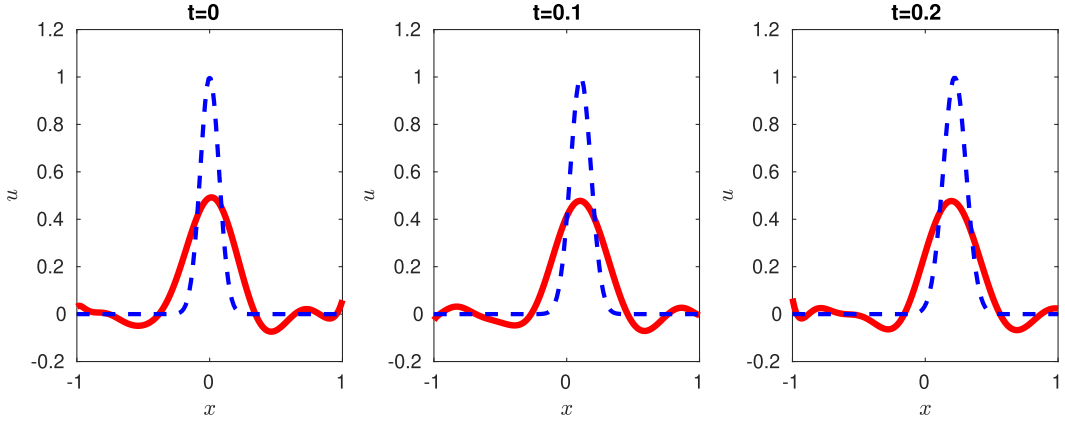


Fig. 20. Exact and PIELM solution of 1D advection of a sharp peaked Gaussian with PIELM. Red: PIELM, Blue: exact.

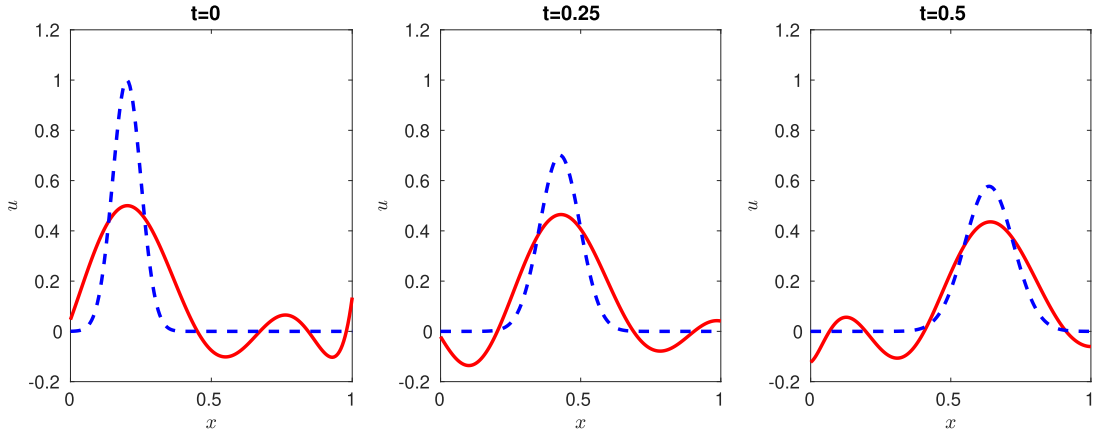
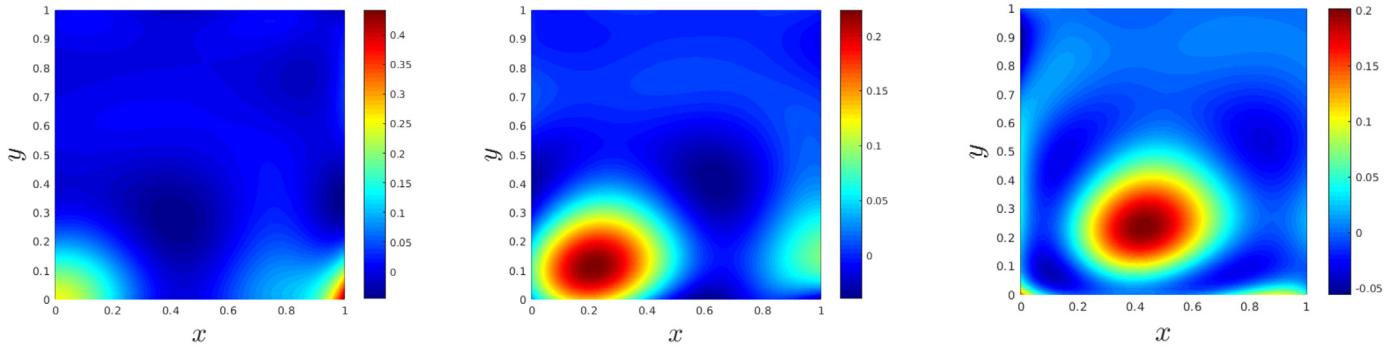


Fig. 21. Exact and PIELM solution for unsteady 1D convection diffusion at $\nu = 0.005$. Red: PIELM, Blue: Exact.



(a) $t = 0$ (b) $t = 0.25$ (c) $t = 0.5$

Fig. 22. PIELM solutions for unsteady 2D convection diffusion at $\nu = 0.005$.

where $\mathcal{N}$ is a linear differential operator and $\partial\Omega\ (= \partial\Omega_L \bigcup \partial\Omega_R)$ is the boundary of computational domain Ω as shown in fig. (24).

6.2. Division of Ω into Finite Elements

In our problem, the rectangular domain Ω is given by $\Omega = (x_L, x_R) \times (0, T)$. On uniformly dividing Ω into N_c non-overlapping rectangular cells, Ω may be rewritten as

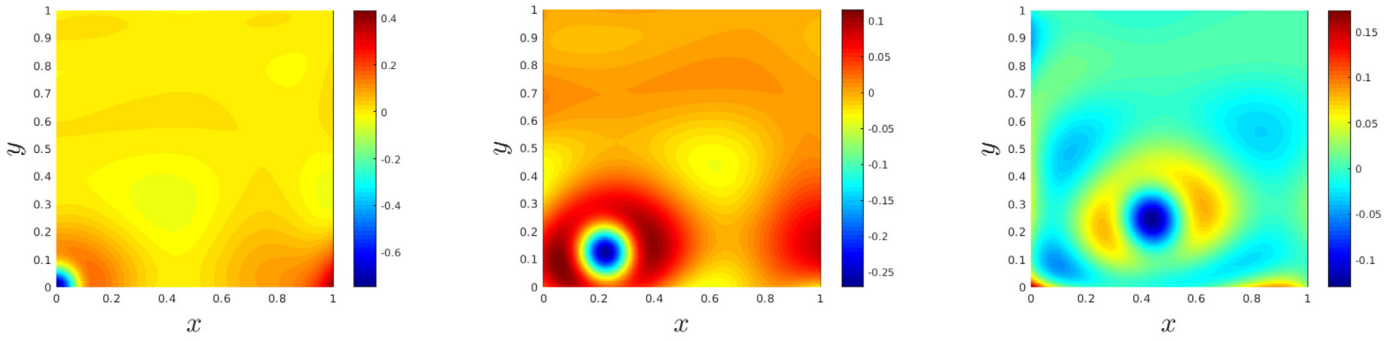
$$\Omega = \bigcup_{i=1}^{N_c} \Omega_i. \tag{66}$$

The boundary of the cell Ω_i is denoted by $\delta\Omega_i$. For rectangular cells,

$$\delta\Omega_i = \bigcup_{m=1}^{4} I_m^{(i)} \tag{67}$$

where $I_m^{(i)}$ represents the m^{th} interface of Ω_i.

Fig. 25 shows the distribution of PIELMs in a rectangular computational domain with $NB_x \times NB_t$ cells (i.e. $N_c = NB_x \times NB_t$). We denote the PIELM on the i^{th} cell by $M^{(i)}$. Fig. 25 also shows an individual PIELM element with collocation points $(\vec{x}_f, \vec{y}_f)^{(i)}$ at the interior (triangles) and the boundary points $(\vec{x}_{bc}, \vec{y}_{bc})^{(i)}$ at the



(a) $t = 0$ (b) $t = 0.25$ (c) $t = 0.5$

Fig. 23. Error contours for unsteady 2D convection diffusion at $\nu = 0.005$.

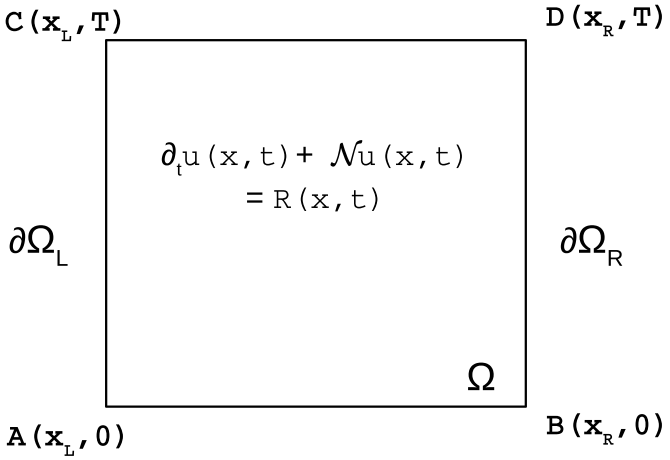


Fig. 24. A diagram of the problem: PDE in a rectangular domain.

four faces (squares). Each individual PIELM has its own unique representation i.e., weights and biases. The output corresponding to a given $M^{(i)}$ is denoted by $f^{(i)}$.

6.3. Interface Regularization

Previously we stated that the distributed network architecture simplifies the representation load of an individual PIELM. However, this type of architecture also gives us an additional task to synchronize all these PIELMs in a physically consistent fashion. To connect all these PIELMs, we introduce additional interface regularization terms to the original cost function. Depending on the known differential operator $\mathcal{N}$, the interface terms may impose soft constraints on continuity or differentiability or a summation of both. For example, continuity at the interface is sufficient for a linear advection equation. However, for the advection-diffusion equation, both continuity and differentiability conditions are required. The inclusion of these terms further constrains the algorithm to ensure consistency of the global solution across PIELM interfaces over the whole domain.

6.4. DPIELM Equations

Now we write the DPIELM equations to be solved for a 1D unsteady problem. Firstly, we will write the equations arising from the original PIELM formulation. Next, we will write additional interface constraints.

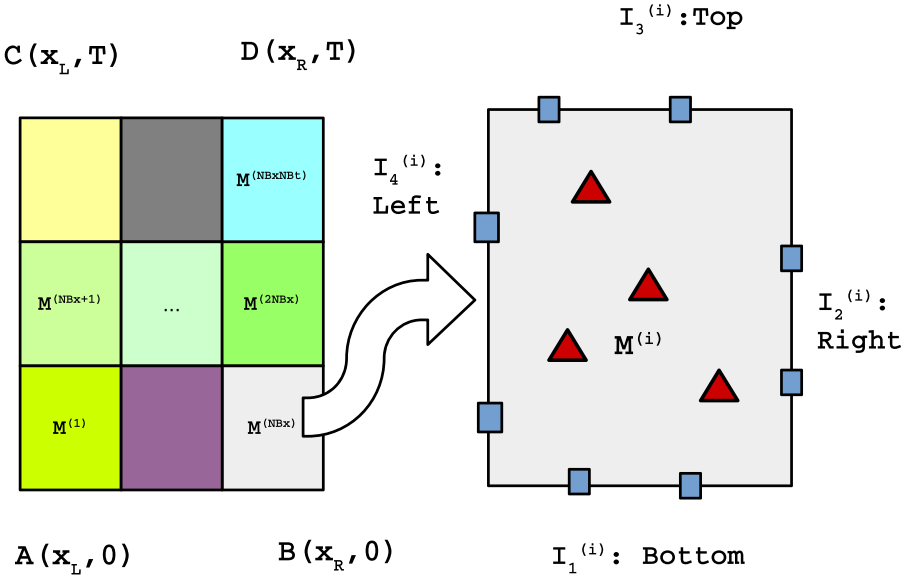


Fig. 25. Left hand side of the figure depicts the uniformly distributed PIELMs in Ω. Right-hand side of the figure shows the exploded view of an individual PIELM element ($i = NB_x$). The blue squares and the red triangles denote the boundary and the collocation points respectively.

6.4.1. Regular PIELM Equations

1. $\overrightarrow{\xi}_{f}^{(i)} = \overrightarrow{0}$

$$\left(\frac{\partial \overrightarrow{f}^{(i)}}{\partial t} + \mathcal{N}\overrightarrow{f}^{(i)} - \overrightarrow{R}^{(i)}\right)_{\Omega_i} = \overrightarrow{0},\ on\ (\overrightarrow{x}_f, \overrightarrow{y}_f)^{(i)}, i = 1, 2, ..., N_c \tag{68}$$

2. $\overrightarrow{\xi}_{bc}^{(i)} = \overrightarrow{0}$

$$(\overrightarrow{f}^{(i)} - \overrightarrow{B}^{(i)})_{I_4} = \overrightarrow{0},\ on\ (\overrightarrow{x}_{bc}, \overrightarrow{y}_{bc})^{(i)}, i = 1, (1 + NB_x), ..., (1 + (NB_t - 1)NB_x) \tag{69}$$

$$(\overrightarrow{f}^{(i)} - \overrightarrow{B}^{(i)})_{I_2} = \overrightarrow{0},\ on\ (\overrightarrow{x}_{bc}, \overrightarrow{y}_{bc})^{(i)}, i = NB_x, 2NB_x, ..., NB_t \times NB_x \tag{70}$$

3. $\overrightarrow{\xi}_{ic}^{(i)} = \overrightarrow{0}$

$$(\overrightarrow{f}^{(i)} - \overrightarrow{F}^{(i)})_{I_1} = \overrightarrow{0},\ on\ (\overrightarrow{x}_f, \overrightarrow{y}_f)^{(i)}, i = 1, 2, ..., NB_x \tag{71}$$

6.4.2. Additional Interface Equations

1. Constraints for C^0 solutions i.e. $\overrightarrow{\xi}_{C^0}^{(i)} = \overrightarrow{0}$.
   - Continuity along x direction

$$\begin{Bmatrix} \overrightarrow{f}^{(\kappa(1)+i-1)} \\ \overrightarrow{f}^{(\kappa(2)+i-1)} \\ ... \\ \overrightarrow{f}^{(\kappa(NB_t)+i-1)} \end{Bmatrix}_{I_2} = \begin{Bmatrix} \overrightarrow{f}^{(\kappa(1)+i)} \\ \overrightarrow{f}^{(\kappa(2)+i)} \\ ... \\ \overrightarrow{f}^{(\kappa(NB_t)+i)} \end{Bmatrix}_{I_4}, i = 1, 2, ..., NB_x - 1 \tag{72}$$

   where $\kappa = [1, (1 + NB_x), ..., 1 + (NB_t - 1)NB_x]^T$.
   - Continuity along t direction

$$\begin{Bmatrix} \overrightarrow{f}^{(\kappa(1)+(i-1)NB_x)} \\ \overrightarrow{f}^{(\kappa(2)+(i-1)NB_x)} \\ ... \\ \overrightarrow{f}^{(\kappa(NB_x)+(i-1)NB_x)} \end{Bmatrix}_{I_3} = \begin{Bmatrix} \overrightarrow{f}^{(\kappa(1)+iNB_x)} \\ \overrightarrow{f}^{(\kappa(2)+iNB_x)} \\ ... \\ \overrightarrow{f}^{(\kappa(NB_x)+iNB_x)} \end{Bmatrix}_{I_1}, i = 1, 2, ..., NB_t - 1 \tag{73}$$

   where $\kappa = [1, 2, ..., NB_x]^T$.
2. Constraints for C^1 solutions i.e. $\overrightarrow{\xi}_{C^1}^{(i)} = \overrightarrow{0}$.
   - Smooth solutions along x direction

$$\frac{\partial}{\partial x}\begin{Bmatrix} \overrightarrow{f}^{(\kappa(1)+i-1)} \\ \overrightarrow{f}^{(\kappa(2)+i-1)} \\ ... \\ \overrightarrow{f}^{(\kappa(NB_t)+i-1)} \end{Bmatrix}_{I_2} = \frac{\partial}{\partial x}\begin{Bmatrix} \overrightarrow{f}^{(\kappa(1)+i)} \\ \overrightarrow{f}^{(\kappa(2)+i)} \\ ... \\ \overrightarrow{f}^{(\kappa(NB_t)+i)} \end{Bmatrix}_{I_4}, i = 1, 2, ..., NB_x - 1 \tag{74}$$

   where $\kappa = [1, (1 + NB_x), ..., 1 + (NB_t - 1)NB_x]^T$.

Assembly of eqns (68 to 74) leads to a system of linear equations which can be represented as

$$\boldsymbol{H}\overrightarrow{c} = \overrightarrow{K}, \tag{75}$$

where $\overrightarrow{c} = [\overrightarrow{c}^{(1)}, \overrightarrow{c}^{(2)}, ..., \overrightarrow{c}^{(N_c)}]^T$. The form of $\boldsymbol{H}$ and $\overrightarrow{K}$ depends on the $\mathcal{N}$, B and F. Finally $\overrightarrow{c}$ can be found using pseudo inverse. It is to be noted that although we have shown the formulation for 1D unsteady problems, no special adjustment is needed to extend the formulation to higher dimensional problems.

The mathematical formulation of DPIELM is complete. The main steps in its implementation are as follows:

1. Divide the computational domain into uniformly distributed non overlapping cells and install a PIELM in each cell.
2. Depending on the cell location, PDE and the initial and boundary conditions, find the expressions for $\overrightarrow{\xi}_{f}^{(i)}$, $\overrightarrow{\xi}_{bc}^{(i)}$ and $\overrightarrow{\xi}_{ic}^{(i)}$ at each cell.
3. Depending on the PDE, find the expressions for $\overrightarrow{\xi}_{C^0}^{(i)}$ and $\overrightarrow{\xi}_{C^1}^{(i)}$ at each cell interface.
4. Assemble these equations in the form of $\boldsymbol{H}\overrightarrow{c} = \overrightarrow{K}$, where $\overrightarrow{c} = [\overrightarrow{c}^{(1)}, \overrightarrow{c}^{(2)}, ..., \overrightarrow{c}^{(N_c)}]^T$.
5. Find the value of $\overrightarrow{c}$ using pseudo inverse.

7. Performance evaluation of DPIELM

7.1. Performance of DPIELM in solving PDEs with sharp gradient solutions

In this Section, we evaluate the performance of DPIELMs by testing it on all the cases in which regular PIELM and PINN failed to perform. The details of the architectures are given in Table (6).

7.1.1. Representation of functions with sharp gradient [TC-9, TC-10]

The mathematical formulation of DPIELM equations for 1D case is as follows:

1. $\overrightarrow{\xi}_{f}^{(i)} = \overrightarrow{0}$

$$\varphi(\boldsymbol{X_f}^{(i)}\boldsymbol{W}^{(i)^T}) \odot \overrightarrow{c}^{(i)} = f(\overrightarrow{x}_f^{(i)}), i = 1, 2, ..., NB_x \tag{76}$$

2. $\overrightarrow{\xi}_{bc}^{(i)} = \overrightarrow{0}$

$$\begin{Bmatrix} \varphi(\boldsymbol{X_{bc,I_1}}^{(1)}\boldsymbol{W}^{(1)^T})\overrightarrow{c}^{(1)} \\ \varphi(\boldsymbol{X_{bc,I_2}}^{(NB_x)}\boldsymbol{W}^{(NB_x)^T})\overrightarrow{c}^{(NB_x)} \end{Bmatrix} = \begin{Bmatrix} f(\overrightarrow{x}_{bc,I_1}^{(1)}) \\ f(\overrightarrow{x}_{bc,I_2}^{(NB_x)}) \end{Bmatrix} \tag{77}$$

   where I_1, I_2 refer to left and right cell interfaces respectively.
3. $\overrightarrow{\xi}_{C^0}^{(i)} = \overrightarrow{0}$

$$\varphi(\boldsymbol{X_{bc,I_2}}^{(i)}\boldsymbol{W}^{(i)^T})\overrightarrow{c}^{(i)} = \varphi(\boldsymbol{X_{bc,I_1}}^{(i+1)}\boldsymbol{W}^{(i+1)^T})\overrightarrow{c}^{(i+1)}, \quad i = 1, 2, ..., NB_x - 1 \tag{78}$$

The equations for the 3D case can be written in a similar fashion. The exact and DPIELM solutions for these cases are shown in figs. 26 and 27 respectively.

7.1.2. 1D steady advection-diffusion equation with low value of diffusion constant [TC-11]

The DPIELM equations are as follows:

- $\overrightarrow{\xi}_{f}^{(i)} = \overrightarrow{0}$

$$\varphi'(\boldsymbol{X_f}^{(i)}\boldsymbol{W}^{(i)^T}) \odot \overrightarrow{c}^{(i)} \odot \overrightarrow{m}^{(i)} - \nu\varphi''(\boldsymbol{X_f}^{(i)}\boldsymbol{W}^{(i)^T}) \odot \overrightarrow{c}^{(i)} \odot \overrightarrow{m}^{(i)} \odot \overrightarrow{m}^{(i)} = \overrightarrow{0}, i = 1, 2, ..., NB_x \tag{79}$$

- $\overrightarrow{\xi}_{bc}^{(i)} = \overrightarrow{0}$

$$\begin{Bmatrix} \varphi(\boldsymbol{X_{bc,I_1}}^{(1)}\boldsymbol{W}^{(1)^T})\overrightarrow{c}^{(1)} \\ \varphi(\boldsymbol{X_{bc,I_2}}^{(NB_x)}\boldsymbol{W}^{(NB_x)^T})\overrightarrow{c}^{(NB_x)} \end{Bmatrix} = \begin{Bmatrix} B(\overrightarrow{x}_{bc,I_1}^{(1)}) \\ B(\overrightarrow{x}_{bc,I_2}^{(NB_x)}) \end{Bmatrix} \tag{80}$$

Table 6
Details of DPIELM architecture for the test cases. For 1D steady problems, architecture is given by $[NB_x, nb_x, N^*_{cell}]$, where NB_x,nb_x and N^*_{cell} refer to number of cells, number of points in the cell and size of hidden layer of the PIELM. Similarly, for 1D and 2D unsteady problems, it is given by $[NB_x, NB_t, nb_x, nb_t, N^*_{cell}]$ and $[NB_x, NB_y, NB_t, nb_x, nb_y, nb_t, N^*_{cell}]$ respectively.

Test Case ID	Description	Dataset and Architecture
TC-9	Representation of 1D sharp and discontinuous functions	[50,5,5]
TC-10	Representation of a sharp peaked 2D Gaussian	[15,15,5,5,15]
TC-11	1D steady advection-diffusion	[20,5,20]
TC-12	Advection of a sharp peaked Gaussian	[15,10,5,5,30]
TC-13	1D unsteady linear advection-diffusion	[10,10,5,5,30]
TC-14	2D unsteady linear advection-diffusion equation	[20,20,50,3,3,3,30]

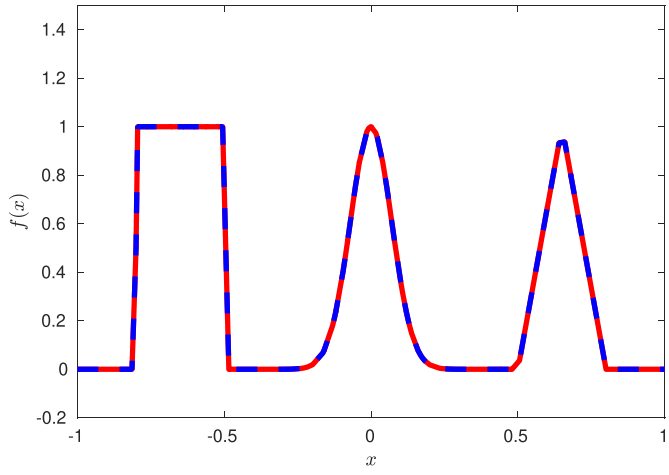


Fig. 26. Representation of 1D non smooth functions with DPIELM. Red: DPIELM, Blue: Exact.

- $\overrightarrow{\xi}^{(i)}_{C^0} = \overrightarrow{0}$

$$\varphi(\boldsymbol{X_{bc,I_2}}^{(i)}\boldsymbol{W}^{(i)^T})\overrightarrow{c}^{(i)} = \varphi(\boldsymbol{X_{bc,I_1}}^{(i+1)}\boldsymbol{W}^{(i+1)^T})\overrightarrow{c}^{(i+1)}, \quad i = 1, 2, ..., NB_x - 1 \tag{81}$$

- $\overrightarrow{\xi}^{(i)}_{C^1} = \overrightarrow{0}$

$$\varphi'(\boldsymbol{X_{bc,I_2}}^{(i)}\boldsymbol{W}^{(i)^T})\overrightarrow{c}^{(i)} \odot \overrightarrow{m}^{(i)} = \varphi'(\boldsymbol{X_{bc,I_1}}^{(i+1)}\boldsymbol{W}^{(i+1)^T})\overrightarrow{c}^{(i+1)} \odot \overrightarrow{m}^{(i+1)}, i = 1, 2, ..., NB_x - 1 \tag{82}$$

The results of the exact and DPIELM solution is given in fig. 28.

7.1.3. 1D unsteady advection of a sharp peaked Gaussian [TC-12]
The DPIELM equation to be solved are as follows:

1. $\overrightarrow{\xi}^{(i)}_{f} = \overrightarrow{0}$

$$\varphi'(\boldsymbol{X_f}^{(i)}\boldsymbol{W}^{(i)^T}) \odot \overrightarrow{c}^{(i)} \odot (\overrightarrow{m}^{(i)} + \overrightarrow{n}^{(i)}) = \overrightarrow{0}, i = 1, 2, ..., N_c \tag{83}$$

2. $\overrightarrow{\xi}^{(i)}_{bc} = \overrightarrow{0}$

$$\varphi(\boldsymbol{X_{bc,I_4}}^{(i)}\boldsymbol{W}^{(i)^T})\overrightarrow{c}^{(i)} = \overrightarrow{0}, i = 1, (1 + NB_x), ..., (1 + (NB_t - 1)NB_x) \tag{84}$$

$$\varphi(\boldsymbol{X_{bc,I_2}}^{(i)}\boldsymbol{W}^{(i)^T})\overrightarrow{c}^{(i)} = \overrightarrow{0}, i = NB_x, 2NB_x, ..., NB_t \text{x} NB_x \tag{85}$$

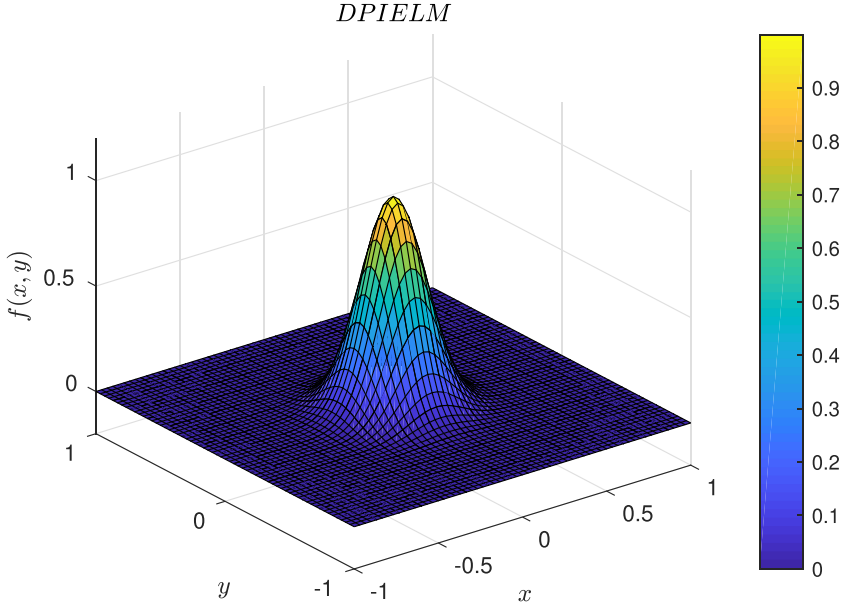


Fig. 27. Representation of a sharp peaked 2D Gaussian with DPIELM

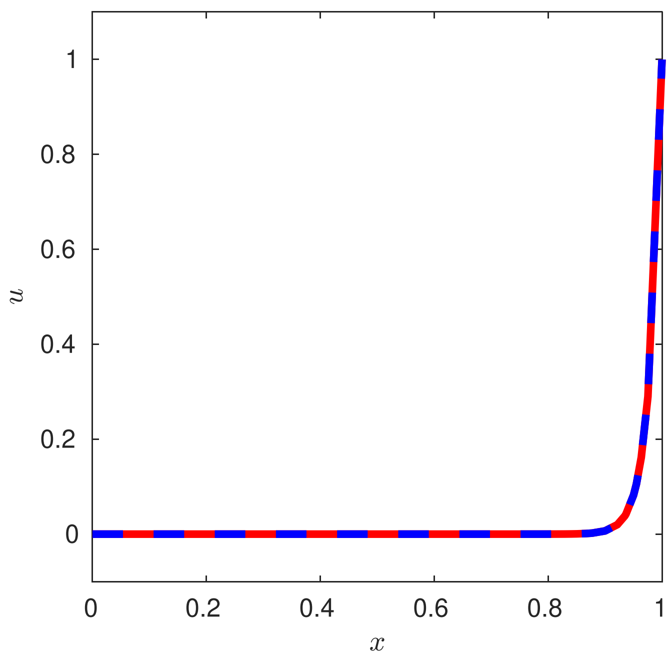


Fig. 28. Exact and DPIELM solution of unsteady 1D convection diffusion. Red: DPIELM, Blue: exact

3. $\vec{\xi}_{ic}^{(i)} = \vec{0}$

$$\varphi(\boldsymbol{X_{bc,I_1}}^{(i)}\boldsymbol{W}^{(i)^T})\vec{c}^{(i)} = \vec{F}(\vec{x}^{(i)}_{bc,I_1}), i = 1, 2, ..., NB_x \tag{86}$$

4. $\vec{\xi}_{C^0}^{(i)} = \vec{0}$
   - Continuity along x direction

$$\begin{Bmatrix} \varphi(\boldsymbol{X_{bc,I_2}}^{(j(1,i)-1)}\boldsymbol{W}^{(j(1,i)-1)^T})\vec{c}^{(j(1,i)-1)} \\ \varphi(\boldsymbol{X_{bc,I_2}}^{(j(2,i)-1)}\boldsymbol{W}^{(j(2,i)-1)^T})\vec{c}^{(j(2,i)-1)} \\ \dots \\ \varphi(\boldsymbol{X_{bc,I_2}}^{(j(NB_t,i)-1)}\boldsymbol{W}^{(j(NB_t,i)-1)^T})\vec{c}^{(j(NB_t,i)-1)} \end{Bmatrix} = \begin{Bmatrix} \varphi(\boldsymbol{X_{bc,I_4}}^{(j(1,i))}\boldsymbol{W}^{(j(1,i))^T})\vec{c}^{(j(1,i))} \\ \varphi(\boldsymbol{X_{bc,I_4}}^{(j(2,i))}\boldsymbol{W}^{(j(2,i))^T})\vec{c}^{(j(2,i))} \\ \dots \\ \varphi(\boldsymbol{X_{bc,I_{I_4}}}^{(j(NB_t,i))}\boldsymbol{W}^{(j(NB_t,i))^T})\vec{c}^{(j(NB_t,i))} \end{Bmatrix} \tag{87}$$

   where $\rho = 1, 2..., NB_t$, $j(\rho, i) = \kappa(\rho) + i$, $i = 1, 2, ..., NB_x - 1$ and $\kappa = [1, (1 + NB_x), ..., 1 + (NB_t - 1)NB_x]^T$.
   - Continuity along t direction

$$\begin{Bmatrix} \varphi(\boldsymbol{X_{bc,I_3}}^{(j(1,i-1)}\boldsymbol{W}^{(j(1,i-1)^T})\vec{c}^{(j(1,i-1)} \\ \varphi(\boldsymbol{X_{bc,I_3}}^{(j(2,i-1)}\boldsymbol{W}^{(j(2,i-1)^T})\vec{c}^{(j(2,i-1)} \\ \dots \\ \varphi(\boldsymbol{X_{bc,I_3}}^{(j(NB_t,i-1)}\boldsymbol{W}^{(j(NB_t,i-1)^T})\vec{c}^{(j(NB_t,i-1)} \end{Bmatrix} = \begin{Bmatrix} \varphi(\boldsymbol{X_{bc,I_1}}^{(j(1,i))}\boldsymbol{W}^{(j(1,i))^T})\vec{c}^{(j(1,i))} \\ \varphi(\boldsymbol{X_{bc,1}}^{(j(2,i))}\boldsymbol{W}^{(j(2,i))^T})\vec{c}^{(j(2,i))} \\ \dots \\ \varphi(\boldsymbol{X_{bc,I_{I_1}}}^{(j(NB_t,i))}\boldsymbol{W}^{(j(NB_t,i))^T})\vec{c}^{(j(NB_t,i))} \end{Bmatrix} \tag{88}$$

   where $\rho = 1, 2..., NB_x$, $j(\rho, i) = \kappa(\rho) + iNB_x$, $i = 1, 2, ..., NB_t - 1$ and $\kappa = [1, 2, ..., NB_x]^T$.

The result for the TC-12 is given in fig. 29 respectively.

7.1.4. 1D and 2D unsteady linear advection-diffusion equation [TC-13 and TC-14]

In this Section, we present the DPIELM equation for the 1D case. The equations for the 2D case can be formulated similarly. The equations to be solved for 1D unsteady advection-diffusion equation are as follows:

1. $\vec{\xi}_f^{(i)} = \vec{0}$

$$\varphi'(\boldsymbol{X_f}^{(i)}\boldsymbol{W}^{(i)^T})\odot\vec{c}^{(i)}\odot(\vec{n}^{(i)} + a\vec{m}^{(i)} - \nu\vec{m}^{(i)}\odot\vec{m}^{(i)}) = \vec{0}, i = 1, 2, ..., N_c \tag{89}$$

2. $\vec{\xi}_{C^1}^{(i)} = \vec{0}$

$$\begin{Bmatrix} \varphi'(\boldsymbol{X_{bc,I_2}}^{(j(1,i)-1)}\boldsymbol{W}^{(j(1,i)-1)^T})\vec{c}^{(j(1,i)-1)}\odot\vec{m}^{(j(1,i)-1)} \\ \varphi'(\boldsymbol{X_{bc,I_2}}^{(j(2,i)-1)}\boldsymbol{W}^{(j(2,i)-1)^T})\vec{c}^{(j(2,i)-1)}\odot\vec{m}^{(j(2,i)-1)} \\ \dots \\ \varphi'(\boldsymbol{X_{bc,I_2}}^{(j(NB_t,i)-1)}\boldsymbol{W}^{(j(NB_t,i)-1)^T})\vec{c}^{(j(NB_t,i)-1)}\odot\vec{m}^{(j(NB_t,i)-1)} \end{Bmatrix} = \begin{Bmatrix} \varphi'(\boldsymbol{X_{bc,I_4}}^{(j(1,i))}\boldsymbol{W}^{(j(1,i))^T})\vec{c}^{(j(1,i))}\odot\vec{m}^{(j(1,i))} \\ \varphi'(\boldsymbol{X_{bc,I_4}}^{(j(2,i))}\boldsymbol{W}^{(j(2,i))^T})\vec{c}^{(j(2,i))}\odot\vec{m}^{(j(2,i))} \\ \dots \\ \varphi'(\boldsymbol{X_{bc,I_{I_4}}}^{(j(NB_t,i))}\boldsymbol{W}^{(j(NB_t,i))^T})\vec{c}^{(j(NB_t,i))}\odot\vec{m}^{(j(NB_t,i))} \end{Bmatrix} \tag{90}$$

   where $\rho = 1, 2..., NB_t$, $j(\rho, i) = \kappa(\rho) + i$, $i = 1, 2, ..., NB_x - 1$ and $\kappa = [1, (1 + NB_x), ..., 1 + (NB_t - 1)NB_x]^T$.

Rest of the equations i.e. $\vec{\xi}_{bc}^{(i)} = \vec{0}$, $\vec{\xi}_{ic}^{(i)} = \vec{0}$ and $\vec{\xi}_{C^0}^{(i)} = \vec{0}$ are same as that used in 1D unsteady linear advection. The results for the 1D and 2D cases are given in fig. 30 and figs. 31 to 32 respectively.

For this test case (TC-14), we referred to Section 5.1 of Borker et al. [30]. For a mesh size of order 10^{-2}, the L_2 norm of both DPIELM and DGLM of second-order element (i.e. $DGLM_2$) is of same order i.e. 10^{-3}. To the best of authors' knowledge, this is the first application of an ELM based algorithm to solve the 2D unsteady advection-diffusion equation in low viscosity regime. It is a very promising result because it demonstrates the capability of DPIELM architecture and opens the door for more challenging problems in the future.

7.2. Performance of DPIELM in advecting a complicated initial profile

In section 5.1, we illustrated the inability of the deep PINN in solving the linear advection equation subjected to a complicated initial condition. Unlike PINN, DPIELM can solve this problem accurately (fig. 33). It is an interesting result that emphasizes the advantage of efficient design of neural network and training algorithm over the practice of adjusting the depth and size of deep neural networks by hit and trial. The details of the dataset and the architecture are as follows: $NB_x = 15, NB_t = 10, nb_x = 5, nb_t = 5$ and $N^*_{cell} = 30$.

7.3. Effect of number of hidden neurons on DPIELM solutions

For a dataset consisting of 100 points i.e., $N = 100$, we changed the size of the hidden layer for different steady 1D cases (TC-1 to TC-3). The result for 1D steady advection case has been shown in fig. 34 for reference. If the function is smooth, then the performance of both PIELM and DPIELM is similar. For the same number of neurons, the error in PIELM, as well as DPIELM, is of order 10^{-3}. Unlike PIELM, increasing the number of neurons beyond the number of equations improves the accuracy of the solution.

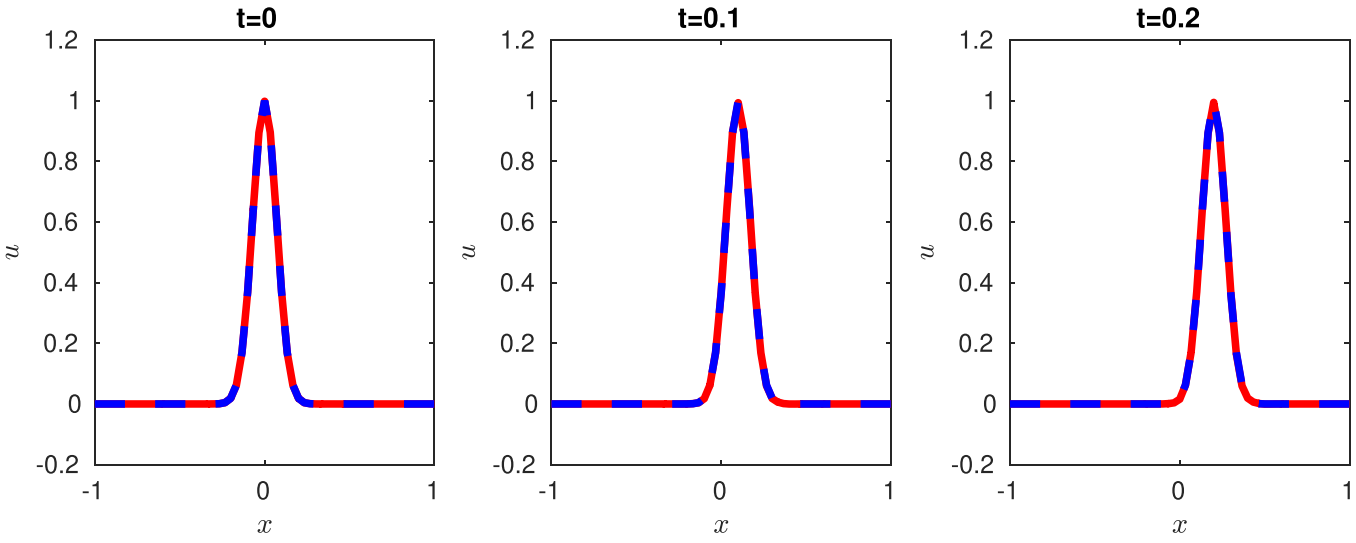


Fig. 29. Exact and DPIELM solution advection of a sharp-peaked Gaussian with DPIELM. Red: DPIELM, Blue: Exact.

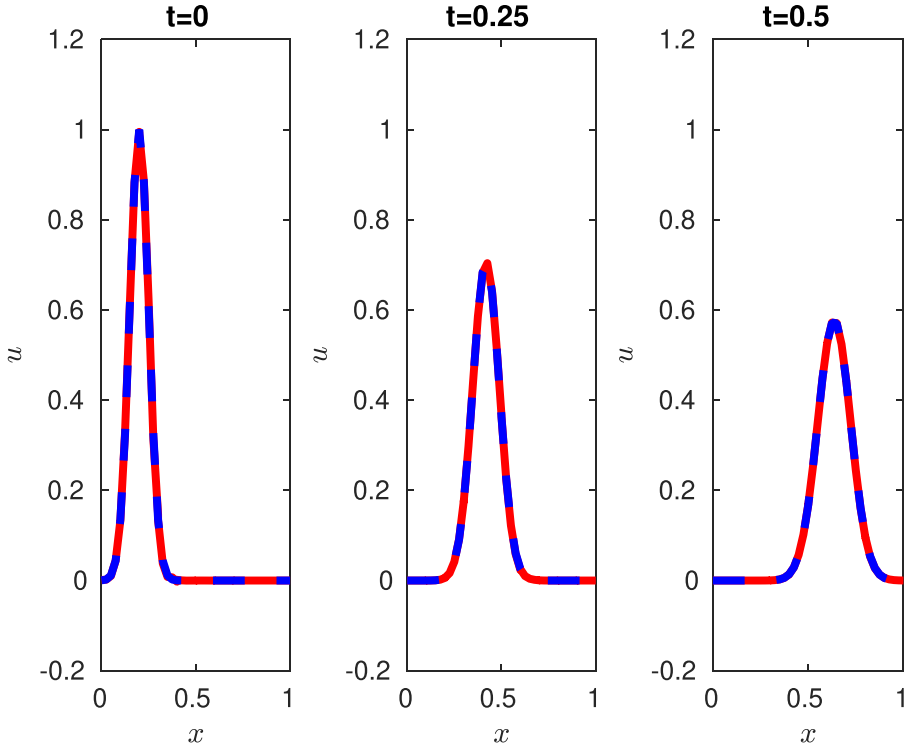


Fig. 30. Exact and DPIELM solution of unsteady 1D convection diffusion at $\nu = 0.005$. Red: DPIELM, Blue: exact

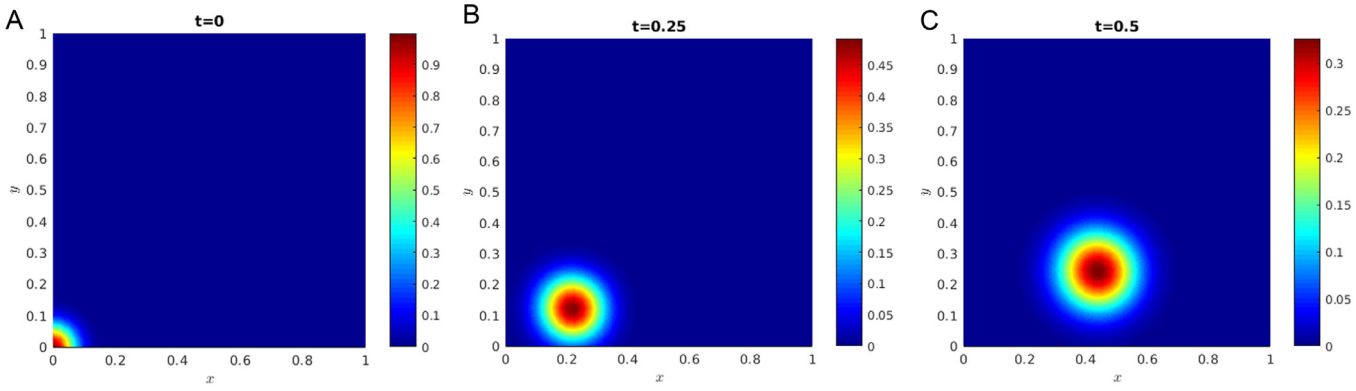


Fig. 31. DPIELM solutions for unsteady 2D convection diffusion at $\nu = 0.005$.

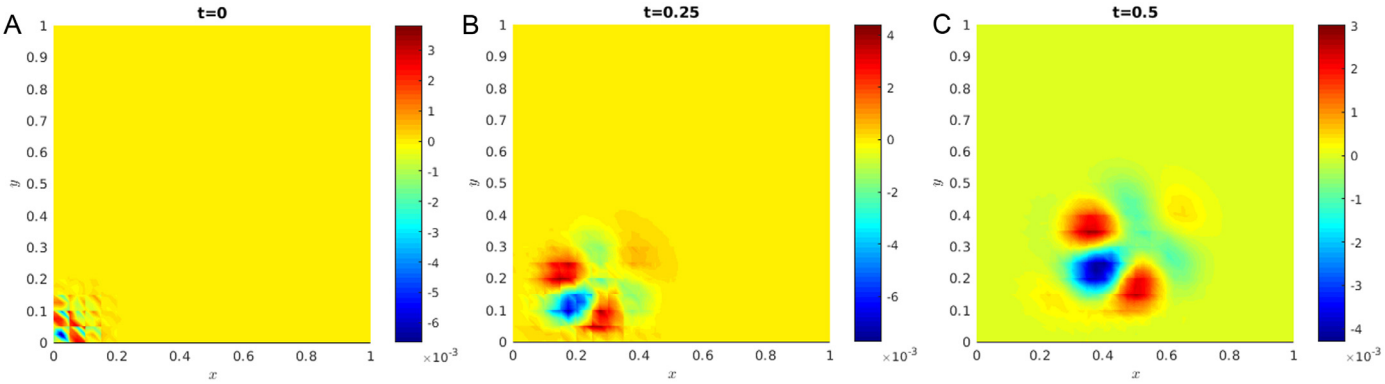


Fig. 32. DPIELM error contours for unsteady 2D convection diffusion at $\nu = 0.005$.

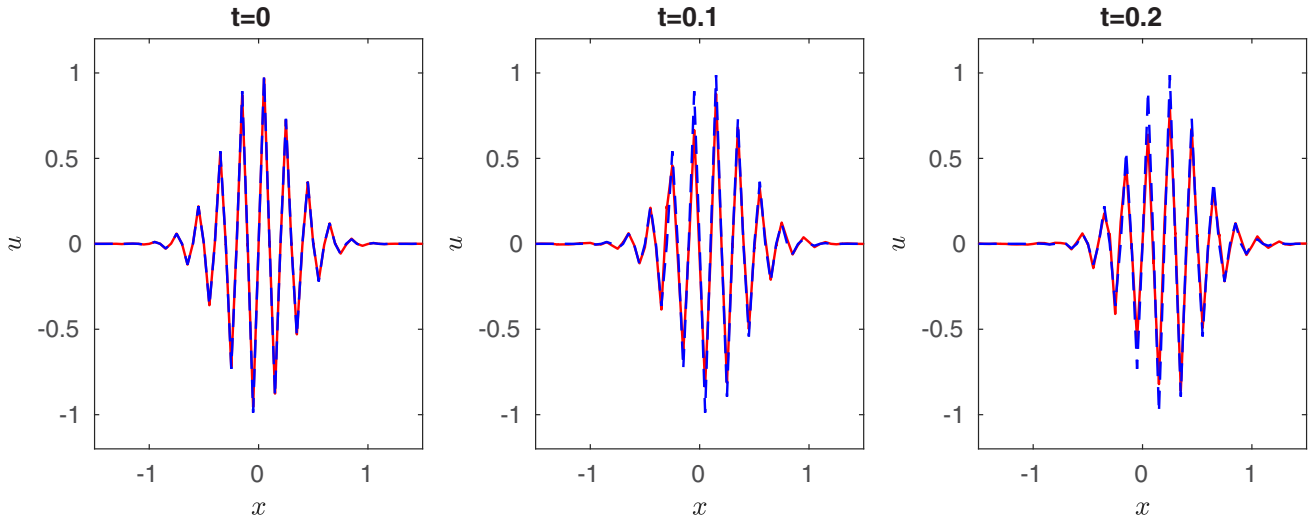


Fig. 33. Exact and DPIELM solution of pure advection of a high-frequency wave packet. Red: DPIELM, Blue: exact

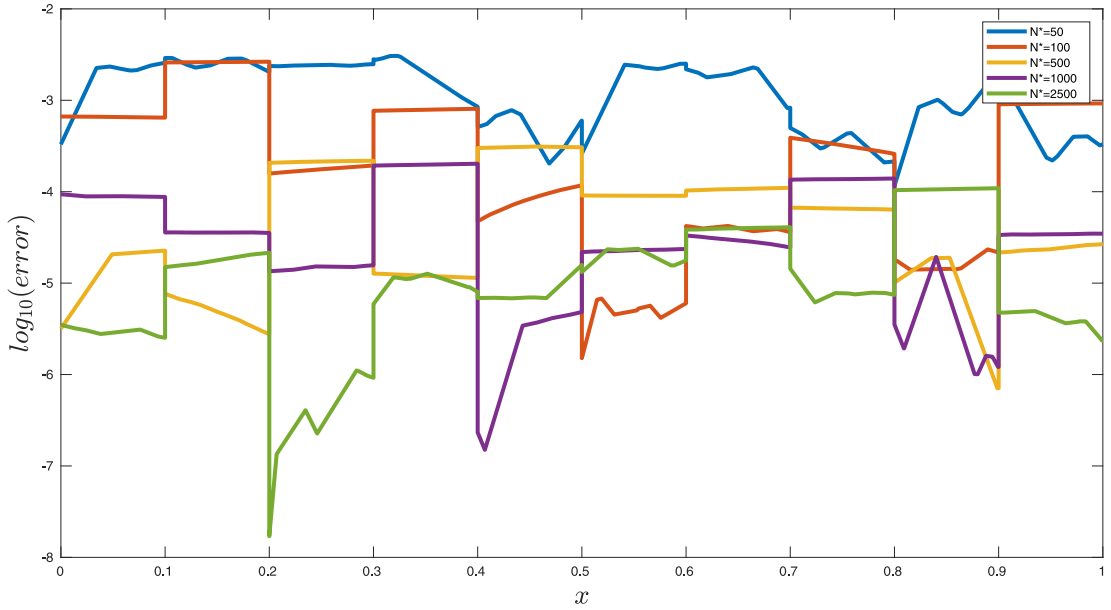


Fig. 34. Effect of number of hidden nodes on DPIELM solution for steady advection case (N=100).

7.4. Summary

In summary, the highlights of DPIELM described so far are as follows:

1. The novel neural network architecture of DPIELM fixes the problem of original PIELM against sharp gradients.
2. Section 7.2 shows a concrete proof of DPIELM's superior representation capacity over existing deep PINN.
3. Section 7.1.4 shows the first application of an ELM based algorithm to solve the 2D unsteady advection-diffusion equation in low viscosity regime.
4. Presently, DPIELM can not handle complex geometries because it's implementation requires the connectivity information among individual cells.

8. Conclusion and future work

We have presented in this paper PIELM – an efficient method to utilize physics informed neural networks to solve stationary and time-dependent linear PDEs. As PIELM inherits the unique qualities of its parent algorithms (PINN and ELM), it works very well on complex geometries, respects the inherent physics of the PDEs, and is extremely fast as well. It leads to several advantages over existing conventional numerical methods. PIELM reduces numerical artefacts such as false diffusion as well can handle complex geometries in a meshfree approach. PIELM also reduces, to a certain extent, the arbitrariness of the number of neurons in typical deep PINNs. Our numerical tests also confirm that, for a fixed problem, our minimal PIELM is more accurate than prior deep NN results [7] while being faster.

We have also presented in this paper the limitations in representing complex functions using a single PIELM or PINN for the whole domain. For practical problems using PINNs can lead to very deep networks with concomitant training problems and efficiency issues. Our proposed solution is to use a distributed PIELM (DPIELM), which uses different representations in different portions of the domain while imposing some continuity and differentiability constraints. The resultant DPIELM easily captures profiles that PINNs have difficulty with. Further, on time-dependent problems, DPIELM gives results that are comparable to sophisticated conventional numerical techniques, as seen in Section 7.1.4.

We believe that the method, as formulated, is already very powerful for linear PDEs with constant or space-varying coefficients. Two primary areas of development remain, in our opinion. Our preliminary tests show that, for linear PDEs, unlike deep PINNs [19], our method may be competitive with conventional techniques in terms of speed and accuracy. However, a firm conclusion on this cannot be reached until a more thorough and fair study is done. Theoretical and numerical evidence for this efficacy is the first area that deserves attention, as the number of practical applications (such as heat conduction, etc.) where numerical methods for linear equations are a staple is large. Having an efficient neural network framework for such problems can be tremendously beneficial to practitioners.

Even more importantly, PIELM' s efficacy is thanks to the linear nature of the final problem. We have, therefore, limited our present study to linear problems. Extending this method to nonlinear equations is the obvious next frontier. It may be approached in one of two ways. The first approach would be to use the PIELM structure as is and then solve the resultant, non-convex optimization problem. Another approach would be to linearize the equation around the current time step to predict a future time step. It would be the equivalent of a linearized, explicit time-stepping method in conventional time marching techniques. We are currently investigating these approaches and will report progress in future publications.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] S.S. Rao, The finite element method in engineering, Butterworth-heinemann, 2017.
[2] R.J. LeVeque, Finite difference methods for ordinary and partial differential equations: steady-state and time-dependent problems, 98, Siam, 2007.
[3] H.K. Versteeg, W. Malalasekera, An introduction to computational fluid dynamics: the finite volume method, Pearson education, 2007.
[4] J.J. Quirk, A contribution to the great riemann solver debate, in: Upwind and High-Resolution Schemes, 1997, pp. 550–569, doi:10.1007/978-3-642-60543-7_22.
[5] G. Cybenko, Approximation by superpositions of a sigmoidal function, Mathematics of Control, Signals and Systems 2 (4) (1989) 303–314, doi:10.1007/BF02551274.
[6] A.G. Baydin, B.A. Pearlmutter, A.A. Radul, J.M. Siskind, Automatic differentiation in machine learning: A survey, J. Mach. Learn. Res. 18 (1) (2017) 5595–5637. URL http://dl.acm.org/citation.cfm?id=3122009.3242010
[7] J. Berg, K. Nyström, A unified deep artificial neural network approach to partial differential equations in complex geometries, Neurocomputing 317 (2018) 28–41, doi:10.1016/j.neucom.2018.06.056.
[8] I.E. Lagaris, A. Likas, D.I. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, IEEE Trans. Neural Netw. 9 (5) (1998) 987–1000, doi:10.1109/72.712178.
[9] I.E. Lagaris, A.C. Likas, D.G. Papageorgiou, Neural-network methods for boundary value problems with irregular boundaries, IEEE Trans. Neural Netw. 11 (5) (2000) 1041–1049, doi:10.1109/72.870037.
[10] B.P. van Milligen, V. Tribaldos, J. Jiménez, Neural network differential equation and plasma equilibrium solver, Phys. Rev. Lett. 75 (20) (1995) 3594, doi:10.1103/PhysRevLett.75.3594.
[11] K.S. McFall, J.R. Mahan, Artificial neural network method for solution of boundary value problems with exact satisfaction of arbitrary boundary conditions, IEEE Trans. Neural Netw. 20 (8) (2009) 1221–1233, doi:10.1109/TNN.2009.2020735.
[12] M. Kumar, N. Yadav, Multilayer perceptrons and radial basis function neural network methods for the solution of differential equations: a survey, Comput Math Appl 62 (10) (2011) 3796–3811, doi:10.1016/j.camwa.2011.09.028.
[13] S. Mall, S. Chakraverty, Application of legendre neural network for solving ordinary differential equations, Appl. Soft Comput. 43 (2016) 347–356, doi:10.1016/j.asoc.2015.10.069.
[14] H. Owhadi, Bayesian numerical homogenization, Multiscale Model Simul 13 (3) (2015) 812–828, doi:10.1137/140974596.
[15] M. Raissi, P. Perdikaris, G.E. Karniadakis, Machine learning of linear differential equations using gaussian processes, J. Comput. Phys. 348 (2017a) 683–693, doi:10.1016/j.jcp.2017.07.050.
[16] M. Raissi, P. Perdikaris, G.E. Karniadakis, Inferring solutions of differential equations using noisy multi-fidelity data, J. Comput. Phys. 335 (2017b) 736–746, doi:10.1016/j.jcp.2017.01.060.
[17] M. Raissi, G.E. Karniadakis, Hidden physics models: Machine learning of nonlinear partial differential equations, J. Comput. Phys. 357 (2018) 125–141, doi:10.1016/j.jcp.2017.11.039.
[18] M. Raissi, P. Perdikaris, G.E. Karniadakis, Numerical gaussian processes for time-dependent and nonlinear partial differential equations, SIAM J SCI COMPUT 40 (1) (2018) A172–A198, doi:10.1137/17M1120762.
[19] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys. 378 (2019) 686–707, doi:10.1016/j.jcp.2018.10.045.
[20] G.-B. Huang, Q.-Y. Zhu, C.-K. Siew, Extreme learning machine: theory and applications, Neurocomputing 70 (1-3) (2006) 489–501, doi:10.1016/j.neucom.2005.12.126.
[21] S. Balasundaram, et al., Application of error minimized extreme learning machine for simultaneous learning of a function and its derivatives, Neurocomputing 74 (16) (2011) 2511–2519, doi:10.1016/j.neucom.2010.12.033.
[22] Y. Yang, M. Hou, J. Luo, A novel improved extreme learning machine algorithm in solving ordinary differential equations by legendre neural network methods, Advances in Difference Equations 2018 (1) (2018) 469, doi:10.1186/s13662-018-1927-x.
[23] H. Sun, M. Hou, Y. Yang, T. Zhang, F. Weng, F. Han, Solving partial differential equation based on bernstein neural network and extreme learning machine algorithm, Neural Processing Letters (2018) 1–20, doi:10.1007/s11063-018-9911-8.
[24] G.-B. Huang, L. Chen, C.K. Siew, et al., Universal approximation using incremental constructive feedforward networks with random hidden nodes, IEEE Trans. Neural Networks 17 (4) (2006) 879–892.

[25] D.E. Rumelhart, G.E. Hinton, R.J. Williams, Learning representations by back-propagating errors, nature 323 (6088) (1986) 533–536.
[26] P.J. Roache, Code Verification by the Method of Manufactured Solutions, Journal of Fluids Engineering 124 (1) (2001) 4–10, doi:10.1115/1.1436090.
[27] D. Kopriva, G. Gassner, An energy stable discontinuous galerkin spectral element discretization for variable coefficient advection problems, SIAM Journal on Scientific Computing 36 (4) (2014) A2076–A2099, doi:10.1137/130928650.
[28] R. Courant, K. Friedrichs, H. Lewy, On the partial difference equations of mathematical physics, IBM Journal of Research and Development 11 (2) (1967) 215–234, doi:10.1147/rd.112.0215.
[29] P. Kim, Matlab deep learning, With Machine Learning, Neural Networks and Artificial Intelligence 130 (2017).
[30] R. Borker, C. Farhat, R. Tezaur, A high-order discontinuous galerkin method for unsteady advection-diffusion problems, Journal of Computational Physics 332 (2017) 520–537, doi:10.1016/j.jcp.2016.12.021.

Vikas Dwivedi received his B.E. degree in Mechanical Engineering from Devi Ahilya Vishwavidyalaya-Indore, India in 2011 and M.Tech degree in Applied Mechanics from Indian Institute of Technology (IIT)-Delhi in 2014. Currently, he is working towards the Ph.D. degree in the Mechanical Engineering Department at IIT-Madras. His research interests include Computational Fluid Dynamics and Applied Machine Learning.

Balaji Srinivasan received his M.S in Aerospace Engineering from Purdue University in 2000 and Ph.D. from the Department of Aeronautics and Astronautics, Stanford University in 2006. He finished his B. Tech in Aerospace Engineering from Indian Institute of Technology (IIT)-Madras in 1998. From 2008-2016, he was serving as a faculty member in the Applied Mechanics Dept at IITDelhi. From 2017, he has been working as an Associate Professor in the Mechanical Engineering Department at IIT-Madras. His research interests include Numerical Analysis, Computational Methods for Partial Differential Equations, Turbulence and Applied Machine Learning.